Etude et mise en place d’une solution pour un WIFI SCANNER avec optimisation de consommation.

Cahier des charges

Durant cette première semaine de stage, je vais prendre connaissance du sujet, prendre en main les équipements, commencer les recherches concernant les commandes AT (nouveauté).

La première étape a été la prise de connaissance du sujet, un sujet tout nouveau pour moi, que ça soit le système de commande AT, le wifi scan, la mise en place d’une solution pour que l’objet consomme le moins possible.

Il s’agit pour l’instant plus d’une prise de note personnelle qu’un rapport que ce soit pour le stage ou l’entreprise, j’ai mis les informations uniquement utiles par 4Mod ici si vous voulez reprendre le sujet/utiliser les infos eu durant le stage sur le module etc

Table des matières

Etude et mise en place d'une solution pour un WIFI SCANNER avec optimisation de consommation. 1

Cahier des charges 3

Identification du module et son état initial 4

Signal, carte SIM, diagnostic et accroche radio 5

Mise a jour firmware et doc WIFI 8

Premiers pas avec les modes d'économie d'énergie 11

Premiers scans Wi-Fi 21

Mesure précise de la consommation selon les modes choisis (eDRX, PSM, Baseline) 23

Base : 23

PSM 26

eDRX 28

Tableau de comparaison 30

Premiers pas sur nRF (cours en ligne) 32

Premières communications entre le microcontrôleur et le module Telit 33

Explication détailler du code (eDRX et Wi-Fi Scan) 33

GNSS 43

Envoi des réponses sur serveur 45

Détection d'un fix valide 47

eDRX Cycle Length 64

Courriel support Telit 65

Cahier des charges de base

Proposer un objet pouvant scanner les WI-Fi environnant

L’objet doit être optimisé au niveau de sa consommation

L’objet doit pouvoir rester connecter au réseau

Tableau comparant les différents modes de faible conso

Mettre en place une communication entre l’objet et un serveur pour l’envoi de données

(Le cahier des charges et susceptible d’évoluer en fonction de l’avancement du sujet.)

Identification du module et son état initial

La première étape de la prise en main a consisté à identifier le matériel avec lequel je vais travailler.

```
AT+CGMM          AT+CGSN
ME310M1-W1       352486280071387

OK               OK
```

Pour commencer la commande AT+CGMM qui a permis de confirmer le modele exact du module (ME310M1-W1), AT+CGSN a permis de récupérer son IMEI (352486280071387) qui est son identifiant unique.

Ce module est connecté avec une evaluation board EVB 2.0, de la marque Telit

```
OK
AT+CGMR
M0U.000002
```

Je vérifie également la version du firmware, information utile a savoir pour la suite.

Ici la version du firmware est la xx2.

Signal, carte SIM, diagnostic et accroche radio

Pour connaitre la qualité du signal je fais AT+CSQ

```
OK
AT+CSQ
+CSQ: 99,99
```

Le signal est de 99.99… Je vérifie sur la doc ce que cela veut dire.

Values:

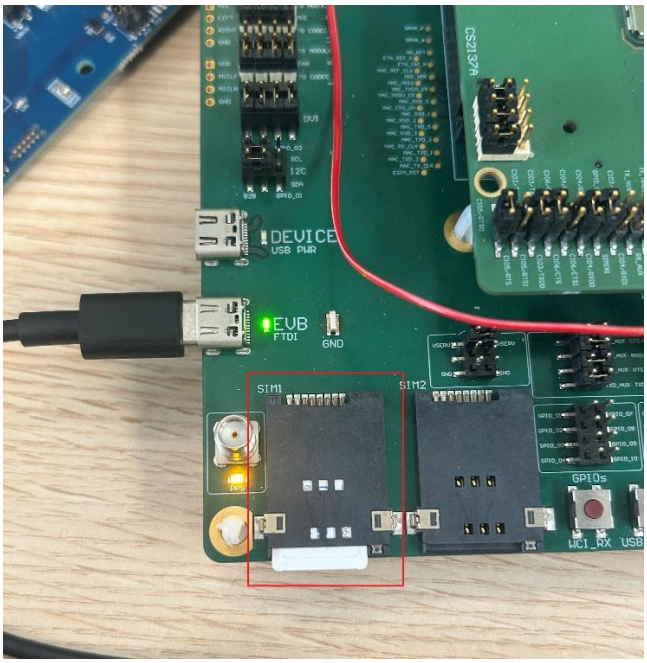
0	:	-113 dBm or less
1	:	-111 dBm
2÷30	:	-109…-53 dBm
31	:	- 51 dBm or greater
99	:	not known or not detectable

La doc dit que si le résultat de cette commande est égal a 99, le signal n'est pas détecté.

Il manque la carte SIM.

```
at+cpin?
+CME ERROR: SIM not inserted
```

Pour pouvoir avancer on me donne une carte SIM de la marque 1nce, je vais faire mes tests dessus.



Je plug/dé-plug l'EVB, il le faut pour que la SIM soit détecté.

```
OK
AT+Cpin?
+CPIN: READY

OK
AT+CSQ
+CSQ: 25,99
```

Apres redémarrage, l'insertion de la carte SIM « 1NCE » a été validé par le status +CPIN: READY.

```
at+CEREG?
+CEREG: 0,5
```

AT+CEREG? Ma permis de connaitre le statut d'enregistrement réseau et m'a donné la valeur 0,5; confirmant que le module est bien enregistré sur une antenne en Roaming, car il s'agit d'une carte SIM multi-opérateurs.

Registered and roaming

When the devices are in cell coverage the packet network end of things always works… in other words AT+CGREG and AT+CEREG always return 0,5 which means **registered and roaming**.

```
OK
AT+COPS?
+COPS: 0,0,"Orange F",7
```

La commande AT+COPS? : le module est connecté au réseau Orange, le paramètre 7 confirme l'utilisation de la technologie d'accès LTE-M.

- 3 = GSM w/EGPRS
- 4 = UTRAN w/HSDPA
- 5 = UTRAN w/HSUPA
- 6 = UTRAN w/HSDPA and HSUPA
- 7 = E-UTRAN
- 8 = EC-GSM-IoT (A/Gb mode)
- 9 = E-UTRAN (NB-S1 mode)

Interpreting this, 0 would typically be 2G connectivity, 7 is typically LTE (including LTE-M), and 9 is typically NB-IoT.

Mise a jour firmware et doc WIFI

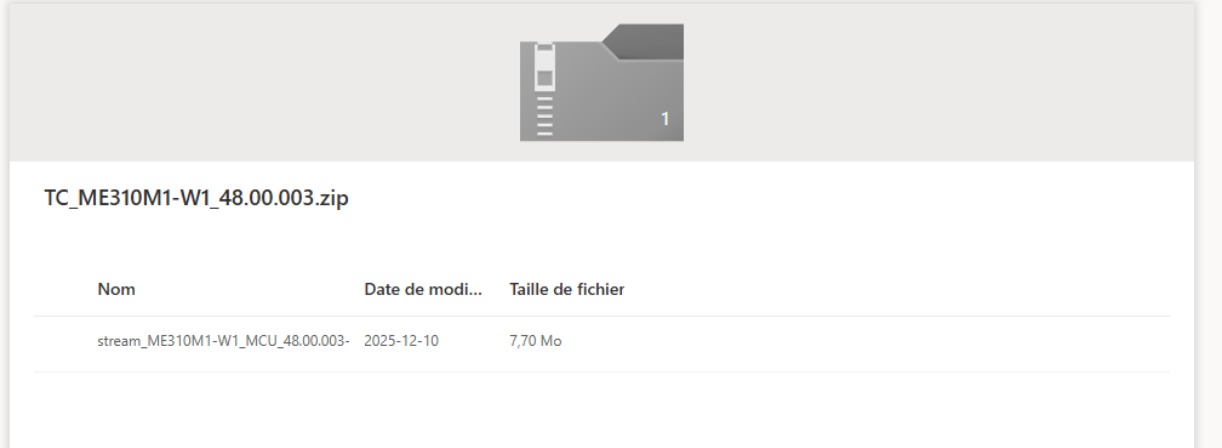
Durant ma première semaine, ça a surtout été des tests de commande, pour être un peu plus a l'aise avec les outils mis a ma disposition pour le stage.

En feuilletant les différentes documentations données par Telit, je ne trouve pas les commandes de wifi scan, en cherchant sur internet je tombe sur une ou deux commandes, qui ne fonctionnent pas.

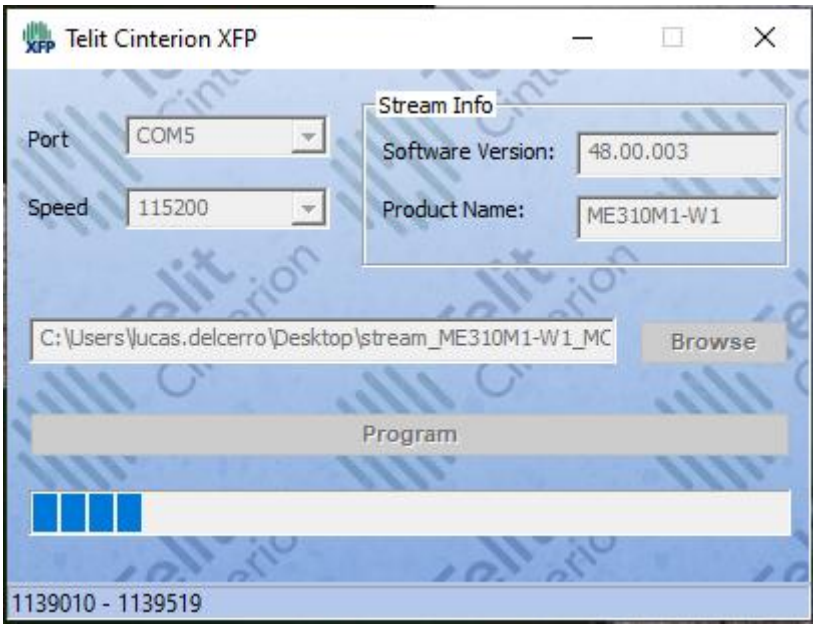
Je contacte mon tuteur qui me pousse un mail m'expliquant que le module doit avoir son firmware en xx3 pour le wifi scan.

> Pour info, check tes versions
>
> **De :** Jerome Martin
>
> Oui en mettant à jour le firmware du module de l'interface 3990252628 .
> L'interface 3990252628 embarque un module avec version xx2 .. il faut donc la mettre à jour vers xx3 pour le wifi scan.
>
> ++
> Jerome.

Comme vu précédemment, ma version du firmware est actuellement la xx2, je vais donc devoir flasher la nouvelle version du firmware pour avoir accès aux outils de wifi scan.



Je flash la nouvelle version du firmware a l'aide de XFP, outil de Telit.



Mise a jour terminé je relance TCATC et je verifie que la version du firmware a bien changé.

```
OK
AT+CGMM
ME310M1-W1

OK
AT+CGMR
M0U.000003
```

La version a bien été mise a jour.

Problème, toujours pas de commandes Wi-Fi Scan fonctionnel..

A l'aide de Benoit, qui envoie un mail au support Telit, on arrive enfin après plusieurs jours a avoir accès a la doc Wi-Fi Scan, doc non disponible publiquement, uniquement sur demande.

[4MOD] ME310M1-W1 - Wifi positionning | Case Number: 00671306

TS-EMEA <ts-emea@telit.com>
À Benoit Gorostidi
Cc Lucas Del Cerro

Répondre | Répondre à tous | Transférer

Cliquez ici pour télécharger des images. Pour protéger la confidentialité, Outlook a empêché le téléchargement automatique de certaines images dans ce message.

80756ST11244A_r0.pdf
323 KB

Hi Benoit,

The ME310M1 modules are calibrated for the WiFi feature from the part number produced with the official version xx2. If you want, with the IMEI, I can check in our database the FW version used in production for your sample. If it's true that the official version xx2 has been applied by us, this sample is ok to test this feature.

The AT commands for the WiFi come from the chipset vendor and are available from the official version xx3. So, you did well to update the Module!
Since the WiFi commands aren't common with other platforms, we decided to document them in a dedicated guide that you can find attached.

Best regards,
Matteo Soccol

EMEA Technical Support
Telit Cinterion Application Engineering

J'ai donc pu faire quelques tests avec les commandes AT, pour l'instant rien de concret, juste de quoi me familiariser avec ce nouveau type de commandes.

Premiers pas avec les modes d'économie d'énergie

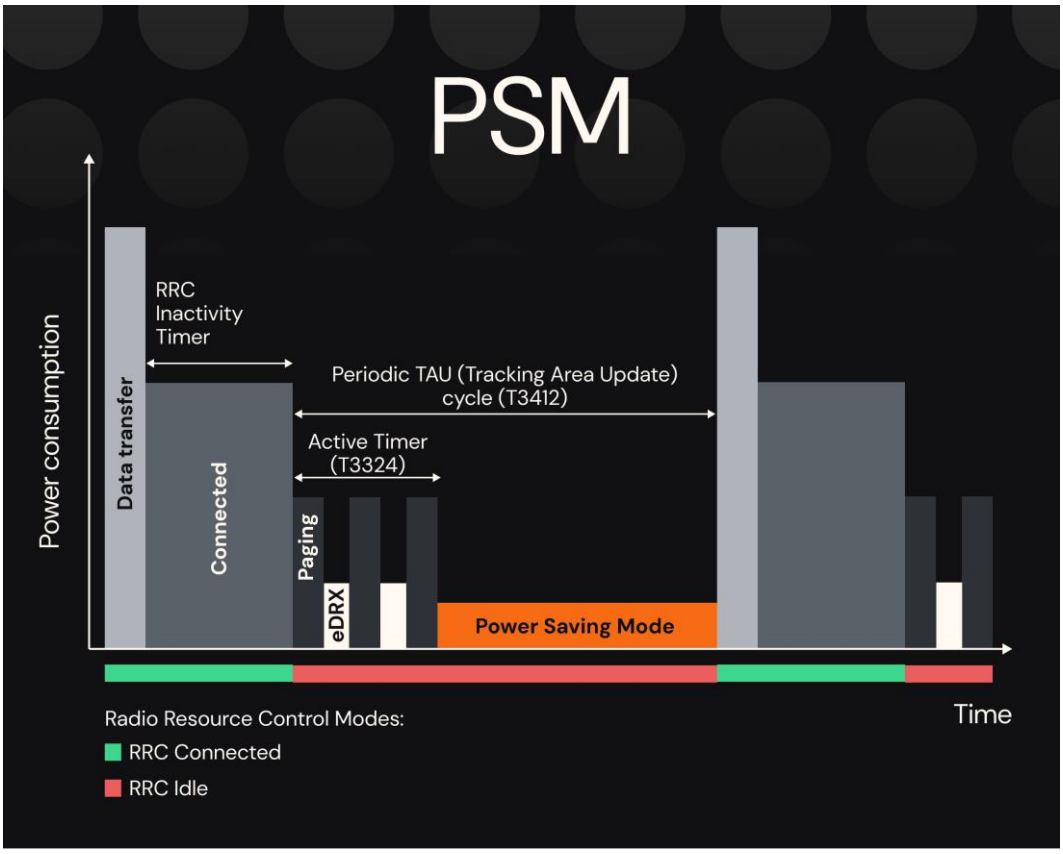
On ma donné pour mission de tester deux types de mode d'économie d'énergie :

- PSM (Power Saving Mode)

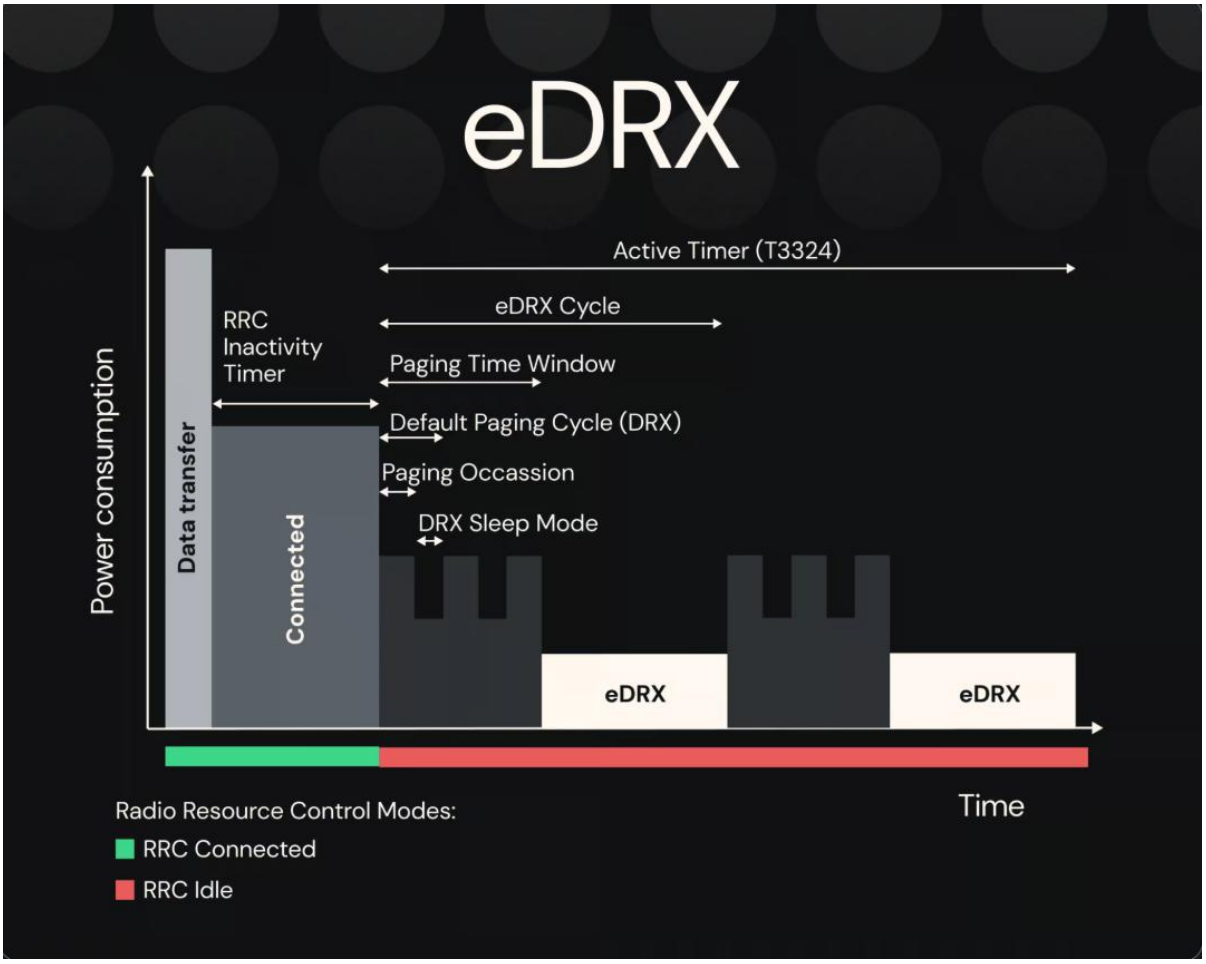
- eDRX (Extended Discontinuous Reception)

Je vais devoir me renseigner sur ces deux modes, voir ce qu'ils font, leurs différences, s'ils sont compatibles avec le cahier des charges, peser le pour et le contre de chacun puis **à termes**, faire un choix.

Le PSM est un mode d'économie d'énergie fourni par le réseau. Lorsqu'il est activé, le module entre dans un état de veille profonde après avoir effectué ses communications. Pendant cette période, il coupe complètement sa partie radio et tombe à quelques µA de consommation. Le module reste techniquement enregistré sur le réseau mais est injoignable depuis l'extérieur — il se réveille uniquement sur timer interne ou signal hardware, souvent ce sont les operateurs qui choisissent le temps de sleep (c'est le cas de 1nce), le PSM, c'est surtout pour les objets IoT qui ont besoin d'etre la 2-3 fois par jours, moins c'est plus compliqué.

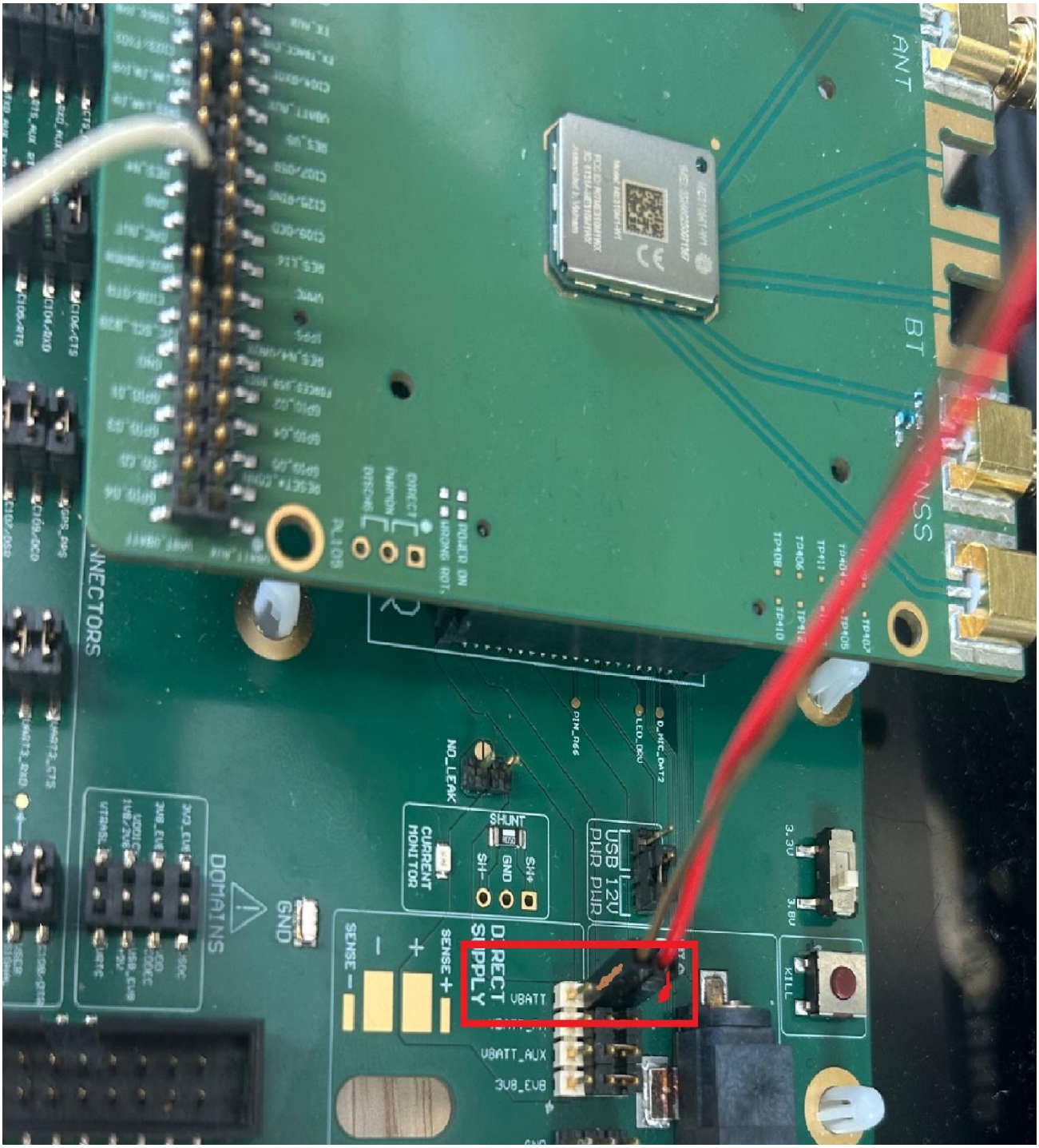


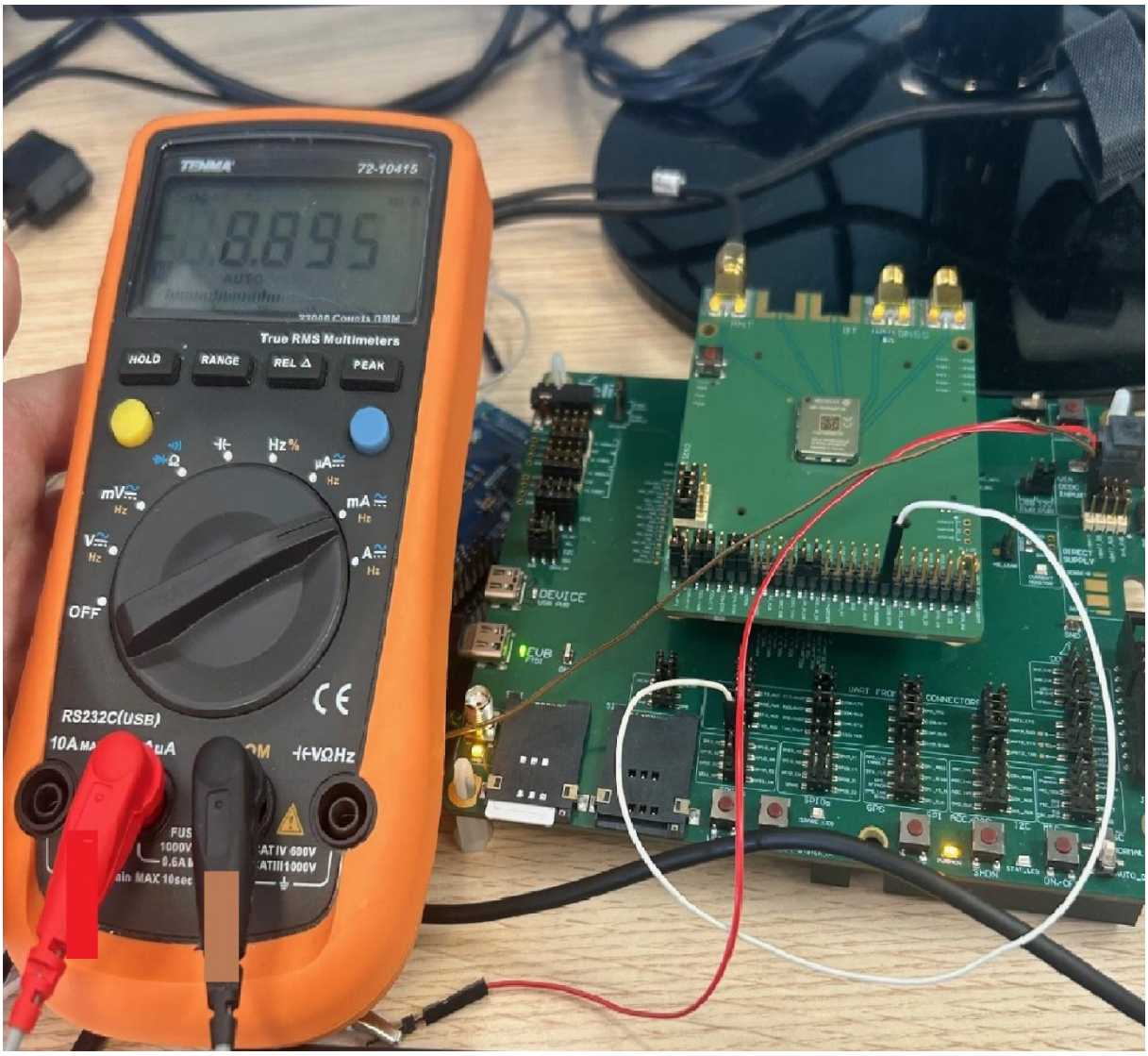
L'eDRX est un mode d'économie d'énergie qui permet au module de ne pas écouter le réseau en permanence. Plutôt que de vérifier continuellement la présence de données entrantes comme en DRX classique, le module alterne entre des phases de sommeil et des fenêtres d'écoute appelées PTW (Paging Time Window). Entre ces fenêtres, le module ne consomme presque rien. Le cycle eDRX est négocié avec le réseau et peut aller de quelques secondes à plusieurs minutes. Pendant tout ce temps, le module reste enregistré sur le réseau et joignable lors des fenêtres d'écoute.



On va maintenant effectuer les mesures pour les différents modes (Baseline, PSM et eDRX)

Câblage pour calculer la consommation du module





Rallier les pins au multimetre, ne pas oublier d'enlever le jumper NO_LEAK « *Remove NO_LEAK jumper if present* » *ME310M1 Power Consumption Application Note*

Test de conso sans mode d'économie d'énergie (baseline)

```
at#CEDRXS=0,4,"0011"
OK
AT#CPSMS=0
OK
```

at#CEDRXS=0,4,"0011" — Désactive eDRX

AT#CPSMS=0 — Désactive le PSM

Conso moyenne entre 10mA et 8mA (a vu d'œil)

C'est beaucoup et pas du tout optimisé pour un objet LTE-M censé être utilisable une longue durée.

Je l'ai ajouté pour avoir une valeur a laquelle se comparer quand je ferais les tests PSM et eDRX.

Mise en place du PSM

En analysant la doc du ME on a pu voir que le bouton wakeup était lié au bouteau « reset » de l'evaluation kit, c'est pour l'instant lui qui me permet de mettre en place le PSM et que le module puisse se réveiller.

```
at+cgdcont=1,"IP","iot.1nce.net"
OK
at#sgact=1,1
#SGACT: 10.38.97.209

OK
AT#CPSMS=1,,,60,30
OK
AT#PSMURC=1
OK
AT#PING="8.8.8.8"
#PING: 01,8.8.8.8,5,117
#PING: 02,8.8.8.8,1,117
#PING: 03,8.8.8.8,2,117
#PING: 04,8.8.8.8,1,117

OK

#PSMURC: 30,4170
```

at+cgdcont=1,"IP","iot.1nce.net" — Configure le contexte PDP avec l'APN 1NCE

at#sgact=1,1 — Active le contexte PDP (connexion données)

AT#CPSMS=1,,,60,30 — Active le PSM avec timer voulu en secondes (pas possible avec cette carte SIM, le temps est aléatoire et supérieur a 3000) et le temps avant de s'endormir (quand il se réveille ou a l'application de la commande)

AT#PSMURC=1 — Active les URCs (notifications) liées aux transitions d'état PSM

AT#PING="8.8.8.8" — Ping DNS google, pas obligatoire mais utile pour voir si le contexte PDP a bien été activé.

#PSMURC : 30,4170 — Apres 30 secondes les URC annonce qu'on est bien passé en PSM, ici pour une durée de 4170 secondes soit ~70minutes de sommeil.

Les résultats :

On peut voir qu'en PSM, lorsque le module « dort » il consomme très peu (0.015mA), quand il se réveille il reprend une conso basique de 8Ma, comme en Baseline.

C'est l'idéal pour ma situation ?

En PSM le module s'endort de manière très profonde et « quitte » le réseau, il est écrit dans le cahier des charges que le module ne doit pas quitter le réseau.

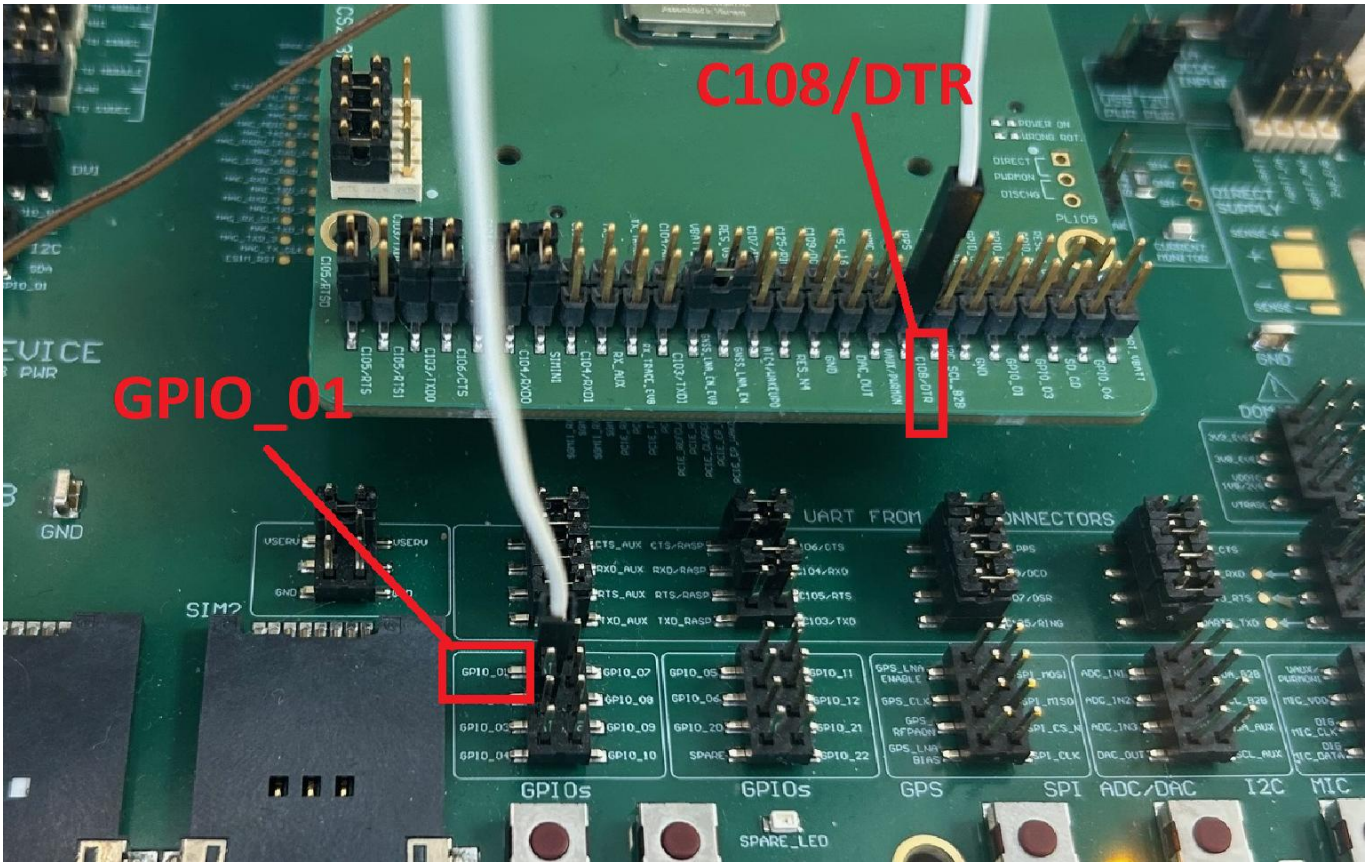
Également on ne peut pas choisir quand le module se réveille avec les commandes, il est possible de le faire en appuyant sur le bouton reset de l'EVB, surement également avec l'uC, mais ça ne règle pas ce problème de réseau.

Mise en place de l'eDRX

Pour optimiser l'économie d'énergie de l'eDRX il faut faire un AT+CFUN=5

"Remember to configure it in the proper way on the ME310M1: the DTR shall be mapped in an available GPIO (e.g. GPIO1 --> AT#GPIO=1,0,9) and then you need to physically connect this GPIO with the DTR of the serial port." **(E-mail du support Telit)**

```
AT#GPIO=1,0,9
OK
```



Retour à TCATC

```
at+cgdcont=1,"IP","iot.1nce.net"
OK
at#sgact=1,1
#SGACT: 10.38.97.209

OK
at#CEDRXS=2,4,"0011"
OK
at+cedrxrdp
+CEDRXRDP: 4,"0011","0011","0000"

OK
AT+CFUN=5
OK
```

at+cgdcont=1,"IP","iot.1nce.net" — Configure le contexte PDP avec l'APN 1NCE

at#sgact=1,1 — Active le contexte PDP (connexion données)

at#CEDRXS=2,4,"0011" — Active eDRX en LTE-M avec un cycle de 40.96 secondes

at#cedrxrdp — Lit les paramètres eDRX négociés avec le réseau (valeurs réelles)

at+cfun=5 —Etat basse consommation (CFUN=5), le ME310M1 est en mode veille, ce qui signifie qu'il est enregistré sur le réseau et qu'il écoute les paginations du réseau

Les résultats :

On peut voir qu'en PSM, lorsque le module « dort » il consomme peu (0.071mA), quand il se réveille pendant quelques ms (a vu d'œil sur le multimètre) il remonte a ~6.5Ma puis replonges dans son sommeil quasi instantanément

C'est l'idéal pour ma situation ?

Pour l'instant c'est la solution que je choisirai (l'oscilloscope sera bien plus précis et c'est lui qui aura le dernier mot.) car tout ce fait tout seul, pas de reset sur l'EVB uniquement des commandes AT a mettre en amont. Grace a CFUN=5 et l'eDRX le module reste connecté au réseau, écoute les paginations du réseau tout en étant « endormi » (idle) et en consommant beaucoup moins que la conso prévue sans mode d'économie d'énergie.

A noté, tout de même, que la conso est tout de même supérieure a celle en eDRX (logique et à voir lors des tests avec l'oscilloscope qui eux implique plusieurs réveils)

Premiers scans Wi-Fi

Maintenant que j'ai la documentation pour les commandes de wifi scan et que j'en sais un peu plus sur les modes de réduction de consommation énergétique je vais pouvoir, a la manière des modes d'économies d'énergie, faire mes premiers scans.

Le wifi scan, qui a pour but de scanner et donner certaines informations sur les Wi-Fi environnants se m'est en place avec plusieurs commandes AT :

AT+CGDCONT=1,"IP","iot.1nce.net"
AT#SGACT=1,1
AT#CEDRXS=2,4,"0011"
AT+CEDRXRDP
AT+CEREG?
AT#PING="8.8.8.8"
AT%WIFICFG="SET","CHANNEL",1,6,11
AT%WIFICFG="SET","TIMEOUT",120,2000
AT%WIFICFG="SET","SCANTABLE",20,0
AT%WIFIEV="REGSCAN",1
AT%WIFICMD="CLEARRES"
AT%WIFICMD="START",0

Attente de la fin du scan puis -> *AT%WIFICMD="GETRES"*

```
AT%WIFICMD="GETRES"
%WIFICMD: "78:C2:13:D7:D4:A0",-62,3790,1,"Livebox-D4A0",1
%WIFICMD: "DE:10:04:E7:EE:C9",-70,3690,6,"Redmi ADU",6
%WIFICMD: "6A:57:3E:64:48:06",-79,3620,6,"SFR_47FF_5GHZ",6
%WIFICMD: "CC:2D:21:83:DC:15",-71,3530,11,"TendalPC_28580A",9
%WIFICMD: "A8:BA:25:85:CB:02",-81,3729,1,"WIFI CLIENT SGS",1
%WIFICMD: "A8:BA:25:85:CB:00",-81,3831,1,"PDAWLAN",1
%WIFICMD: "38:C0:EA:7E:95:78",-44,3632,6,"4MOD-Guest",6
```

Le wifi-scan se comporte bien, a noté que, étant dans les locaux de 4MOD, le scan peut ne pas être pousser a son maximum, durant les vrais tests (ceux quand tout sera techniquement fonctionnel) je serais dehors pour pouvoir en scanner le plus possible. Actuellement on nous prouve que les commandes fonctionne et que les wifi environnants sont bien scannés.

Je vais prendre l'exemple de la « Livebox-D4A0 » pour expliquer les résultats

%WIFICMD: "78:C2:13:D7:D4:A0",-62,3790,1,"Livebox-D4A0",1

"78:C2:13:D7:D4:A0" : MAC unique du point d'accès détecté.

%WIFICMD: "78:C2:13:D7:D4:A0",-62,3790,1,"Livebox-D4A0",1

-62 : C'est le RSSI, la force du signal en dBm

%WIFICMD: "78:C2:13:D7:D4:A0",-62,3790,1,"Livebox-D4A0",1

3790 : C'est le timestamp, le moment du scan (en ms)où ce réseau a été capté.

%WIFICMD: "78:C2:13:D7:D4:A0",-62,3790,1,"Livebox-D4A0",1

1 : C'est le canal Wi-Fi sur lequel le réseau émet.

%WIFICMD: "78:C2:13:D7:D4:A0",-62,3790,1,"Livebox-D4A0",1

"Livebox-D4A0" : C'est le SSID, le nom du Wi-Fi.

Le Wi-Fi Scan fonctionne bien, il me donne bien toutes les informations souhaité concertant les wifi environnant

Mesure précise de la consommation selon les modes choisis (eDRX, PSM, Baseline)

Des premières mesures sur un oscilloscope ont été faites.

Les tests étaient les suivant :

-Une capture de 3 minutes avec un seul PING a 1m30

-Une capture de 10 minutes avec un ping a 4min et un autre a 8min.

Voici ce que les tests ont donné

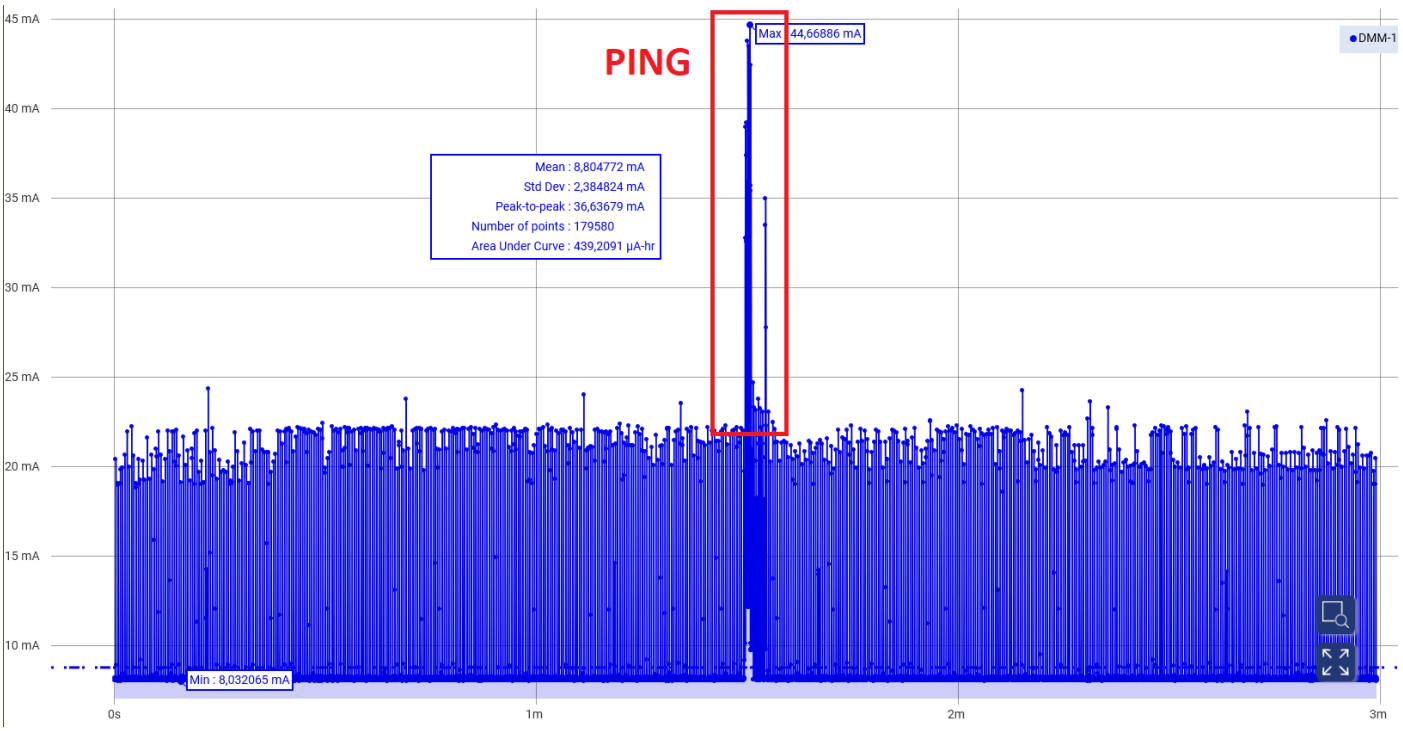
Les mesures ont été effectué garce au multimetre et le l'application Kickstart.

Base :

Pour commencer, la première capture a été celle sans aucun mode d'économie d'énergie, en « Baseline »

Conso 3min

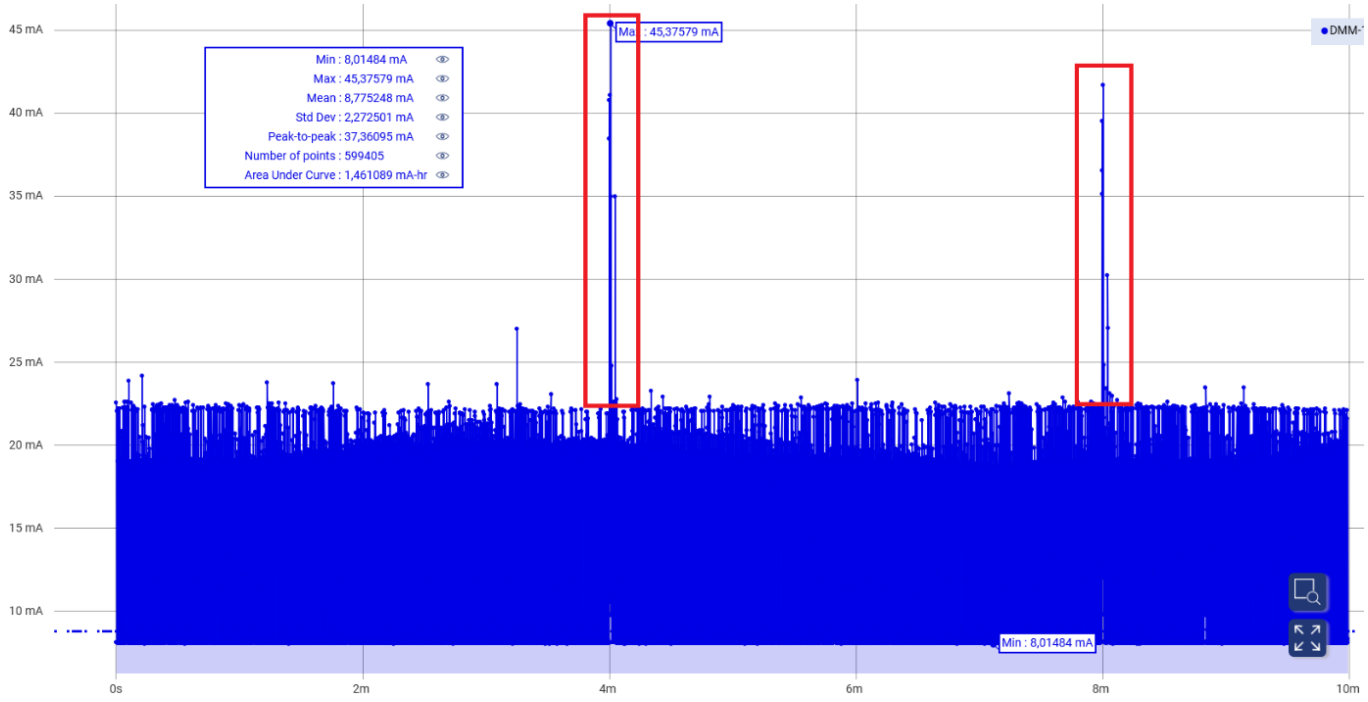
Rien a faire dans TCATC si ce n'est envoyer le ping a 1m30



Min	0,000000	8,0320653e-3
Max	179,579000	44,668858e-3
Mean	89,789500	8,8047718e-3
StdDev	51,840425	2,3848244e-3
CV	57,74 %	27,09 %

La moyenne est de ~8.80mA

Conso 10 minutes (2 ping)



Min	0,000000	8,0148404e-3
Max	599,404000	45,375793e-3
Mean	299,702000	8,7752483e-3
StdDev	173,033463	2,2725006e-3
CV	57,74 %	25,90 %

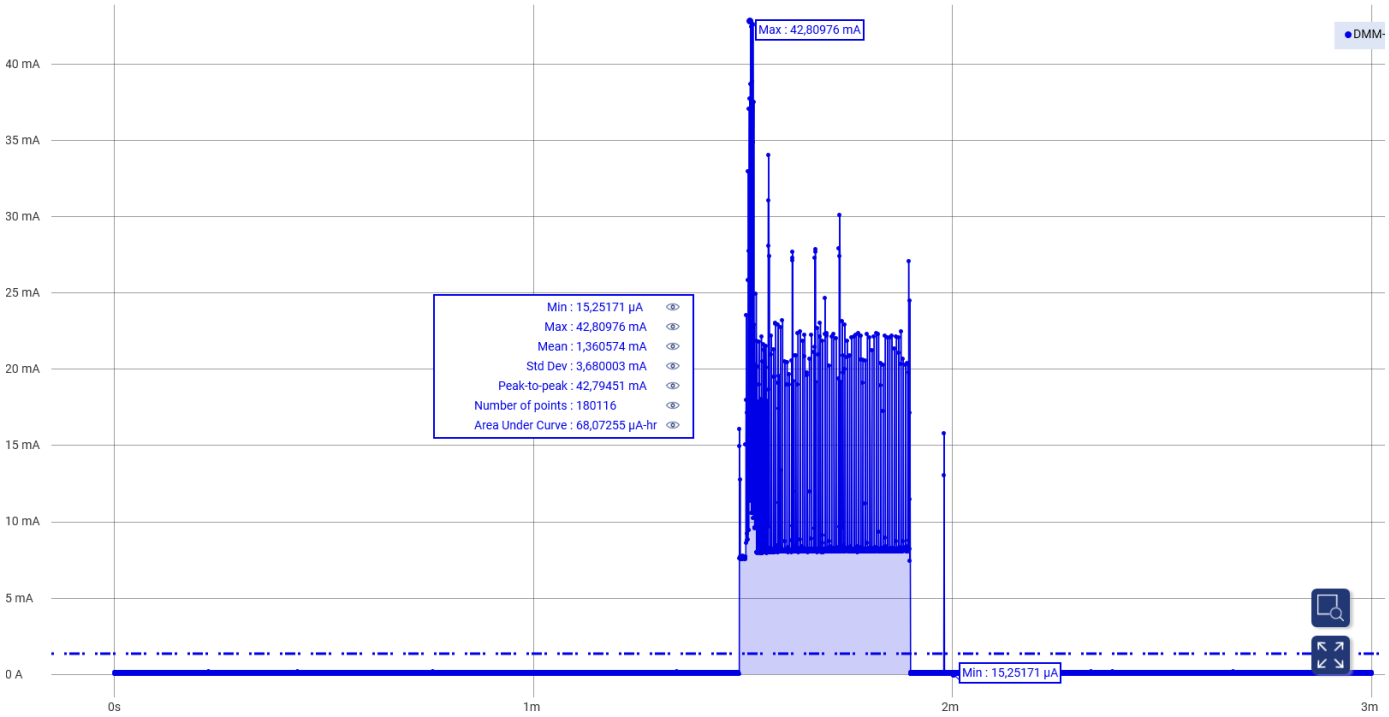
La moyenne est de ~8.77

On savait que la consommation allait être haute sans aucun mode d'économie d'énergie, c'est pour avoir une base de comparaison avec les deux autres modes.

PSM

Conso 3min

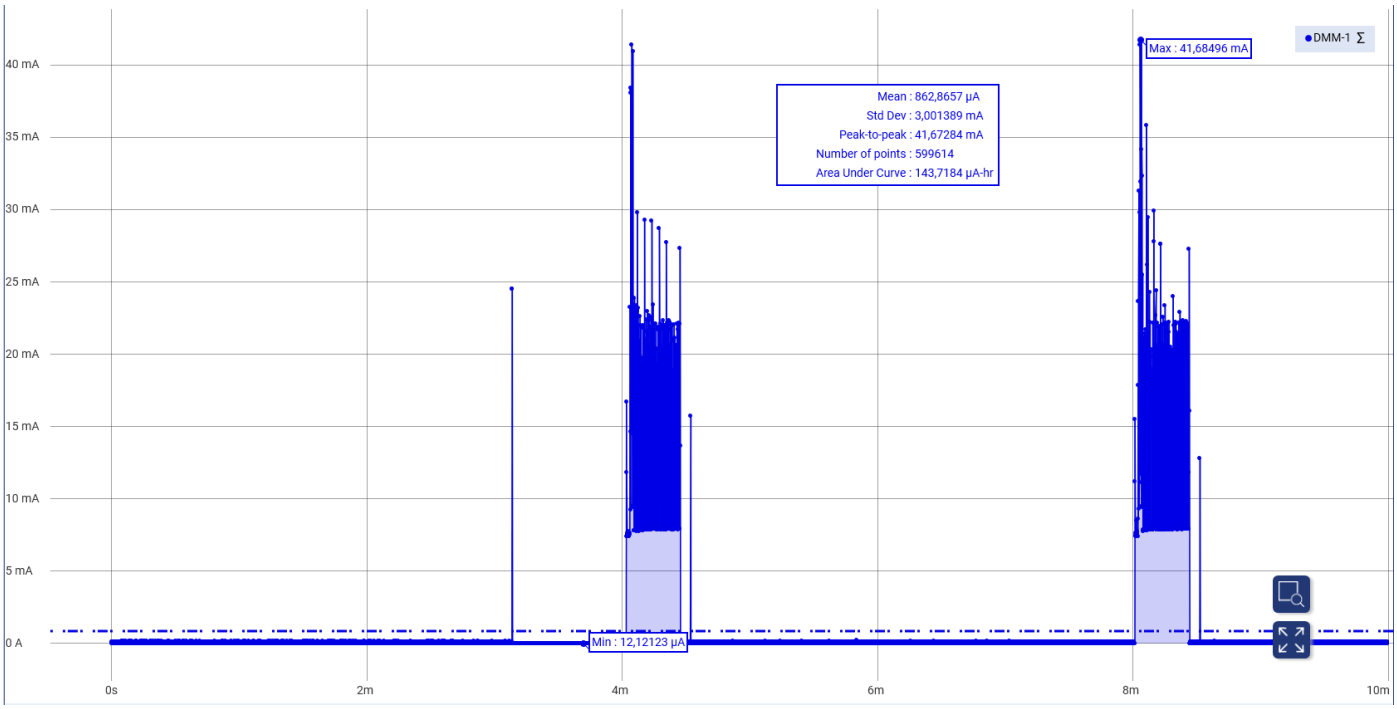
Pour le PSM il y a, car pas encore de uC, des manipulations à faire pour le bon fonctionnement de la capture (WAKE-UP hardware a 1:30 grâce au bouton reset de l'EVB, ping instantanément, demande d'activation PSM avec un délai d'attente de 20 secondes imposés par l'opérateur)



Min	0,000000	15,251710e-6
Max	180,115000	42,809765e-3
Mean	90,057500	1,3605743e-3
StdDev	51,995155	3,6800034e-3
CV	57,74 %	270,47 %

La moyenne est de ~1.36mA

Conso 10 minutes (2 ping)



Min	0,000000	12,121227e-6
Max	599,613000	41,684965e-3
Mean	299,806500	862,86567e-6
StdDev	173,093796	3,0013892e-3
CV	57,74 %	347,84 %

La moyenne est de ~862µA (0.862Ma)

On remarque une baisse déjà significative de la moyenne de consommation, montrant qu'il y a VRAI intérêt a comparer, et chercher le meilleur mode d'économie d'énergie.

On peut aussi voir qu'après le ping, le passage en sleep ne se fait pas instantanément, il y a un minimum imposé par le fournisseur. Il y en a un également un pour le temps de sleep, qui lui est beaucoup plus conséquent (20 secondes vs 7000secondes en moyenne) pour cela qu'il faut utiliser le WAKE-UP hardware.

Qui plus est une des règles du « cahier des charges » était de était de toujours rester connecter au réseau, le PSM endort quasiment totalement le module, donc il se déconnecte du réseau.

eDRX

Conso 3min

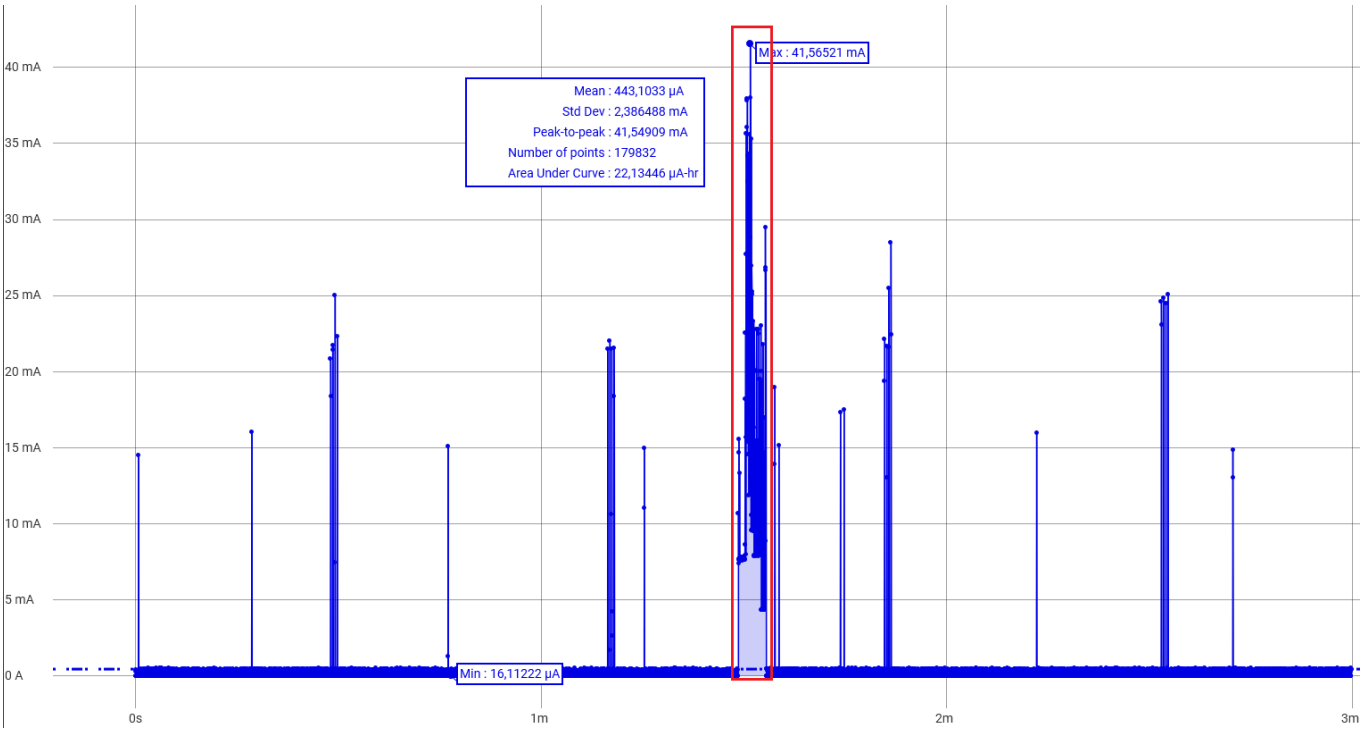
Pour la capture de la conso en mode eDRX, il y a également des manipulations a faire sur TCATC car pas encore de uC, grâce au mail du support Telit, on sait qu'on peut « jouer » avec le bouton DTR de TCATC ce qui permet de faire fonctionner le CFUN et donc de rendre efficace l'eDRX.

"--> UE registered, DTR ON;

--> enable eDRX if it was disabled

--> AT+CFUN=5"

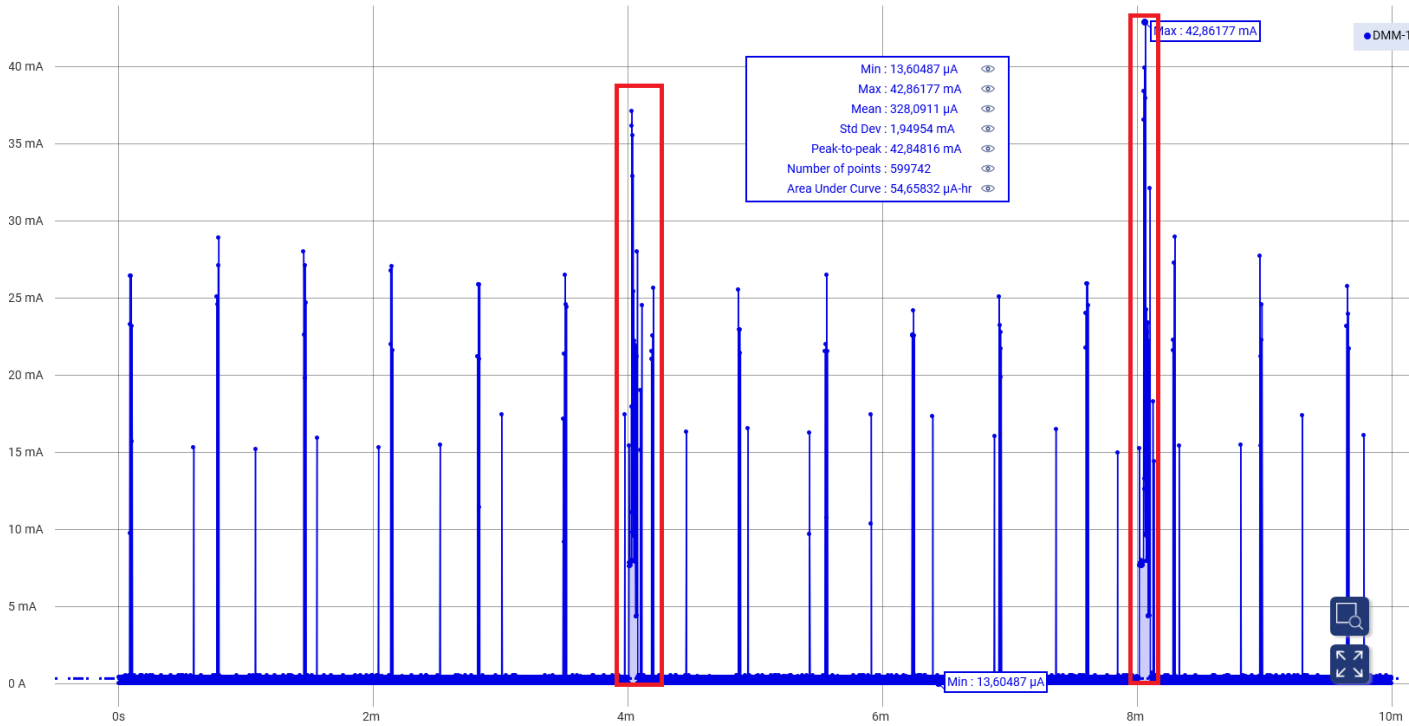
Il faut mettre DTR sur OFF lorsque l'on veut envoyer une commande AT, on pourra automatiser cela plus tard sur le microcontrôleur



Min	0,000000	16,112222e-6
Max	179,831000	41,565206e-3
Mean	89,915500	443,10325e-6
StdDev	51,913171	2,3864883e-3
CV	57,74 %	538,59 %

La moyenne est de ~443µA (0.443Ma)

Conso 10 minutes (2 ping)



Min	0,000000	13,604868e-6
Max	599,741000	42,861766e-3
Mean	299,870500	328,09114e-6
StdDev	173,130747	1,9495400e-3
CV	57,74 %	594,21 %

C’est en eDRX que la moyenne est la plus basse, car avec l’eDRX on se remet quand on veut et instantanément en mode « sleep » sans être bloqué par l’opérateur de la carte SIM.

L’eDRX endort le module mais pas non plus au point du PSM (c’est la raison pour laquelle que le PSM consomme quand même moins que l’eDRX quand il est en sleep, c’est intéressant, on verra pour quel contexte.)

Maintenant il faut choisir, **eDRX ou PSM ?**

Tableau de comparaison

Mode	eDRX	PSM
Consommation moy. 3 min (1 ping)	~443 µA	~1,36 mA
Consommation moy. 10 min (2 pings)	~328 µA	~862 µA
Réduction vs Baseline (3 min)	-95%	-85%
Réduction vs Baseline (10 min)	-96%	-90%
Maintien connexion réseau	Oui (fenêtres PTW)	Non (déconnexion complète)
Conformité cahier des charges	Oui	Non
Passage en sleep après ping	Instantané	Délai minimum imposé par l'opérateur
Durée de sleep imposée	Non	Oui mais pas un gros pb
Profondeur du sleep	Léger (écoute réseau périodique)	Profond (module quasi éteint)
Mode retenu	✓	✕

L'eDRX est le mode retenu pour ce projet IoT pour les raisons suivantes :

1. Connexion réseau permanente : contrairement au PSM qui déconnecte le module du réseau, l'eDRX maintient l'enregistrement réseau et respecte l'exigence du cahier des charges.

2. Meilleure consommation sur les mesures réelles : sur 10 minutes (scenario 2 pings), l'eDRX atteint ~328 uA de moyenne, soit une réduction de 96% vs Baseline et 62% vs PSM sur ce scenario.

3. Pas de contrainte operateur : le passage en sleep se fait instantanément après l'envoi, sans délai minimum impose par 1NCE/Orange (contrairement au PSM qui impose ~7000 s de sleep minimum).

4. Facilite d'automatisation : la gestion du DTR et de la commande AT+CFUN=5 sera facilement automatisable sur le microcontrôleur.

Le PSM reste pertinent dans un contexte ou l'objet n'a pas besoin de rester connecte en permanence et ou des cycles de sommeil de plusieurs heures sont acceptables (ex. traceur d'actifs, capteur météo très basse frequence). Pour notre objet, l’EDRX est clairement une option plus viable.

Premiers pas sur nRF (cours en ligne)

J'ai pu effectuer plusieurs cours sur le DevAcademy **Nordic Semiconductor** pour me familiariser avec :

Firmware Development

nRF Connect SDK Fundamentals

★ **9.1**/10 (6073) **18706** already enrolled

Introduction aux bases du SDK nRF Connect pour le développement de firmware sur les microcontrôleurs Nordic.

Product Overview

nRF54L Series Express Course

★ **9.1**/10 (521) **1477** already enrolled

Présentation rapide de la série nRF54L et de ses principales caractéristiques et usages.

Bluetooth Low Energy

Bluetooth Low Energy Fundamentals

★ **9.1**/10 (2824) **9347** already enrolled

Découverte des principes fondamentaux du Bluetooth Low Energy et de son fonctionnement.

Les cours 1 et 2 m'ont été très utile, moi qui n'avais jamais touché a un seul de ces objets jusqu'à maintenant.
Le cours 3 était également utile et plaisant a faire, je ne sais pas s'il va me servir pour ce projet mais j'ai appris et surtout pu me fidéliser avec nRF.

Premières communications entre le microcontrôleur et le module Telit

À l'issue de cette phase de formation, j'ai pu démarrer les premiers tests de communication entre le microcontrôleur nRF et mon module.
L'objectif de cette étape était de valider la communication série UART entre le microcontrôleur et le module, ainsi que l'envoi et la réception de commandes AT simples.

[image de mon code qui fait uniquement un AT}]

Explication détaillée du code (eDRX et Wi-Fi Scan)

1. Inclusions et constantes

```
#include <zephyr/kernel.h>
#include <zephyr/device.h>
#include <zephyr/drivers/uart.h>
#include <zephyr/drivers/gpio.h>
#include <string.h>
#include <stdio.h>

#define TELIT_UART_NODE  DT_NODELABEL(uart21)
#define BUF_SIZE         256
#define DTR_PIN          11      /* P1.11 -> EVB GPIO01 -> module C108/DTR */
#define NB_SCANS         1       /* nombre de scans WiFi par cycle         */
#define SLEEP_MS         300000  /* durée de veille : 5 minutes            */
#define SLEEP_MIN        (SLEEP_MS / 60000)
```

Les #include importent les bibliothèques du RTOS Zephyr nécessaires au projet : gestion du noyau temps réel, des périphériques, de l'UART et des GPIO.

Les #define déclarent les constantes du programme :

- TELIT_UART_NODE : identifiant du port UART utilisé pour communiquer avec le module Telit, tel que défini dans le Device Tree de la carte
- BUF_SIZE : taille des tampons de réception (256 octets), suffisante pour une ligne de réponse AT
- DTR_PIN : numéro de broche GPIO connectée au signal DTR du module, permettant de le mettre en veille
- NB_SCANS : nombre de scans WiFi par cycle
- SLEEP_MS / SLEEP_MIN : durée de veille entre deux cycles (5 minutes)

---

2. Variables globales

```c
K_MSGQ_DEFINE(telit_msgq, BUF_SIZE, 16, 4);

static const struct device *telit_uart = DEVICE_DT_GET(TELIT_UART_NODE);
static const struct device *gpio1_dev  = DEVICE_DT_GET(DT_NODELABEL(gpio1));

static char rx_buf[BUF_SIZE];
static int  rx_buf_pos;
```

K_MSGQ_DEFINE crée une file de messages Zephyr (telit_msgq).

telit_uart et gpio1_dev sont des pointeurs vers les périphériques UART et GPIO, récupérés depuis le Device Tree au moment de la compilation.

rx_buf est un tampon intermédiaire qui accumule les octets reçus caractère par caractère jusqu'à former une ligne complète. rx_buf_pos indique la position courante dans ce tampon.

---

3. Réception UART — interruption telit_rx_cb

```c
static void telit_rx_cb(const struct device *dev, void *user_data)
{
    uint8_t c;

    if (!uart_irq_update(dev))   return;
    if (!uart_irq_rx_ready(dev)) return;

    while (uart_fifo_read(dev, &c, 1) == 1) {
        if ((c == '\n' || c == '\r') && rx_buf_pos > 0) {
            rx_buf[rx_buf_pos] = '\0';
            k_msgq_put(&telit_msgq, rx_buf, K_NO_WAIT);
            rx_buf_pos = 0;
        } else if (c != '\n' && c != '\r' && rx_buf_pos < BUF_SIZE - 1) {
            rx_buf[rx_buf_pos++] = c;
        }
    }
}
```

Cette fonction est déclenchée automatiquement par le matériel dès qu'un octet arrive sur l'UART.

Elle lit les octets un par un depuis le FIFO matériel et les accumule dans rx_buf. Dès qu'un caractère de fin de ligne (\n ou \r) est reçu, la ligne complète est placée dans la file de messages telit_msgq via k_msgq_put, puis le tampon est réinitialisé.

---

4. Envoi d'une commande AT — send_at

```c
/* — Envoi d'une commande AT (polling TX + CR) ———————————— */
static void send_at(const char *cmd)
{
    printk("> %s\n", cmd);
    for (int i = 0; cmd[i]; i++)
        uart_poll_out(telit_uart, cmd[i]);
    uart_poll_out(telit_uart, '\r');
}
```

Cette fonction envoie une commande AT au module Telit via uart_poll_out (mode polling, le processeur attend que chaque octet soit transmis). Un retour chariot \r est ajouté en fin de commande, conformément au protocole AT.

La commande est également affichée dans le log série du nRF (préfixée de >) pour faciliter le débogage.

---

5. Affichage d'une réponse — print_line

```c
/* — Affichage d'une ligne de réponse, résultats WiFi en cyan ——— */
static void print_line(const char *line)
{
    if (strstr(line, "%WIFICMD:") != NULL) {
        printk("\033[36m  [WIFI] %s\033[0m\n", line);
    } else {
        printk("< %s\n", line);
    }
}
```

Affiche une ligne de réponse reçue du module. Les lignes contenant %WIFICMD: sont les résultats de scan WiFi)

---

6. Attente d'une réponse — wait_for

```c
static int wait_for(const char *expected, int timeout_ms)
{
    char line[BUF_SIZE];
    int64_t end = k_uptime_get() + timeout_ms;

    while (k_uptime_get() < end) {
        if (k_msgq_get(&telit_msgq, line, K_MSEC(100)) == 0) {
            print_line(line);
            if (expected && strstr(line, expected)) {
                return 0;
            }
        }
    }
    return -ETIMEDOUT;
}
```

Cette fonction attend qu'une ligne contenant la chaîne expected soit reçue du module, avec un délai maximum (timeout_ms). Elle lit la file de messages en boucle, affiche chaque ligne reçue, et retourne 0 dès que la chaîne attendue est trouvée. Si le délai est dépassé, elle retourne -ETIMEDOUT.

Elle est utilisée systématiquement après chaque send_at pour attendre la réponse OK du module avant de continuer.

7. Vidage de la file — drain_queue

```c
/* — Vide toutes les lignes en attente (dump résultats GETRES) ———— */
static void drain_queue(int extra_ms)
{
    char line[BUF_SIZE];

    k_msleep(extra_ms);
    while (k_msgq_get(&telit_msgq, line, K_NO_WAIT) == 0) {
        print_line(line);
    }
}
```

Attend un délai supplémentaire (extra_ms) pour laisser le module finir d'envoyer toutes ses données, puis vide et affiche toutes les lignes en attente dans la file. Utilisée après la commande GETRES pour récupérer l'intégralité des résultats du scan Wi-Fi.

---

8. Contrôle DTR — dtr_set

```c
/* ── Contrôle DTR : HIGH = module actif, LOW = module en veille ─────── */
static void dtr_set(int val)
{
    gpio_pin_set(gpio1_dev, DTR_PIN, !val); /* actif bas : réveil=LOW, veille=HIGH */
    printk("[DTR] %s\n", val ? "HIGH (actif)" : "LOW  (veille)");
}
```

Pilote le signal DTR connecté à la broche P1.11 du nRF. La logique est inversée (actif bas) : pour mettre le module en veille, la broche est mise en HIGH ; pour le réveiller, elle est mise à LOW. Cette inversion est gérée par le !val passé à gpio_pin_set.

---

9. Calcul et affichage de l'heure du prochain scan — print_next_scan_time

```c
static void print_next_scan_time(void)
{
    char line[BUF_SIZE];
    int64_t end = k_uptime_get() + 3000;
    int hh = -1, mm = -1;

    send_at("AT+CCLK?");
    while (k_uptime_get() < end) {
        if (k_msgq_get(&telit_msgq, line, K_MSEC(100)) == 0) {
            /* Format réponse : +CCLK: "yy/MM/dd,hh:mm:ss±zz" */
            char *comma = strchr(line, ',');
            if (comma && sscanf(comma + 1, "%d:%d", &hh, &mm) == 2 && hh >= 0) {
                /* Ajoute la durée de veille pour obtenir l'heure du prochain scan */
                mm += SLEEP_MIN;
                hh  = (hh + mm / 60) % 24;
                mm %= 60;
                printk(">>> Prochain scan vers %02d:%02d\n", hh, mm);
            } else {
                print_line(line);
            }
        }
    }
}
```

Interroge le module via AT+CCLK? pour obtenir l'heure courante synchronisée par le réseau LTE (NITZ). La réponse est au format "yy/MM/dd,hh:mm:ss±zz". L'heure et les minutes sont extraites avec sscanf, puis la durée de veille (SLEEP_MIN) est ajoutée pour calculer et afficher l'heure approximative du prochain scan.

---

10. Cycle de scan WiFi — wifi_scan_cycle

```c
/* — Un cycle de scan WiFi complet (NB_SCANS passages) ———————— */
static void wifi_scan_cycle(void)
{
    printk("=== Cycle WiFi : %d scans ===\n", NB_SCANS);

    /* Vide les résultats précédents avant de commencer */
    send_at("AT%WIFICMD=\"CLEARRES\"");
    wait_for("OK", 3000);

    /* Lance NB_SCANS scans successifs, les résultats s'accumulent */
    for (int i = 0; i < NB_SCANS; i++) {
        printk("--- Scan %d/%d ---\n", i + 1, NB_SCANS);
        send_at("AT%WIFICMD=\"START\",0");
        /* Attend l'événement REGSCAN qui signale la fin du scan */
        if (wait_for("WIFIEVU", 5000) != 0) {
            printk("Attention : WIFIEVU non reçu pour scan %d\n", i + 1);
        }
    }

    /* Récupère tous les réseaux trouvés (cumul des NB_SCANS passages) */
    send_at("AT%WIFICMD=\"GETRES\"");
    drain_queue(1000);

    /* Remet le tableau de résultats à zéro pour le prochain cycle */
    send_at("AT%WIFICMD=\"CLEARRES\"");
    wait_for("OK", 2000);
}
```

Réalise un cycle complet de détection WiFi en quatre étapes :

1. **CLEARRES** — efface les résultats du cycle précédent
2. **START** — lance le scan ; le module envoie WIFIEVU quand il est terminé
3. **GETRES** — récupère et affiche la liste des réseaux détectés (BSSID, SSID, RSSI)
4. **CLEARRES** — remet le tableau à zéro pour le prochain cycle

---

11. main — Initialisation

```c
int main(void)
{
    /* --- Init GPIO : DTR démarre HIGH (module actif) --- */
    if (!device_is_ready(gpio1_dev)) {
        printk("GPIO1 non prêt\n");
        return -1;
    }
    gpio_pin_configure(gpio1_dev, DTR_PIN, GPIO_OUTPUT_LOW);
    dtr_set(1);

    /* --- Init UART --- */
    if (!device_is_ready(telit_uart)) {
        printk("UART non prêt\n");
        return -1;
    }
    uart_irq_callback_user_data_set(telit_uart, telit_rx_cb, NULL);
    uart_irq_rx_enable(telit_uart);

    k_msleep(5000); /* attente démarrage du module */
```

La fonction main commence par vérifier que les périphériques GPIO et UART sont disponibles, puis les initialise :

- La broche DTR est configurée en sortie et mise à HIGH (module actif)
- Le callback d'interruption UART telit_rx_cb est enregistré et les interruptions de réception sont activées
- Une pause de 5 secondes laisse le module Telit terminer son démarrage

---

12. main — Configuration AT initiale

```c
    k_msleep(5000); /* attente démarrage du module */

    /* --- Vérification basique --- */
    send_at("AT");
    wait_for("OK", 3000);

    /* --- Activation des messages d'erreur verbeux --- */
    send_at("AT+CMEE=2");
    wait_for("OK", 3000);

    /* --- Mappage GPIO1 du module sur le signal DTR (ALT8 = DTR sur GPIO_01) --- */
    send_at("AT#GPIO=1,0,9");
    wait_for("OK", 3000);

    /* --- Configuration de l'APN --- */
    send_at("AT+CGDCONT=1,\"IP\",\"iot.1nce.net\"");
    wait_for("OK", 5000);

    /* --- Activation du contexte PDP (ignoré si déjà actif) --- */
    send_at("AT#SGACT?");
    if (wait_for("#SGACT: 1,1", 3000) != 0) {
        send_at("AT#SGACT=1,1");
        if (wait_for("#SGACT:", 30000) != 0) {
            printk("Attention : timeout SGACT\n");
        }
        wait_for("OK", 5000);
    } else {
        printk("Contexte PDP déjà actif\n");
        wait_for("OK", 1000);
    }
```

```c
    send_at("AT+CTZU=1");
    wait_for("OK", 3000);
    send_at("AT#NITZ=7");
    wait_for("OK", 3000);

    /* --- Activation eDRX : CAT-M (act-type=4), cycle ~10.24s (0011) --- */
    send_at("AT#CEDRXS=2,4,\"0011\"");
    wait_for("OK", 3000);

    /* --- Configuration WiFi unique (persiste entre les cycles de veille) --- */
    send_at("AT%WIFICFG=\"SET\",\"CHANNEL\",1,6,11");
    wait_for("OK", 3000);
    send_at("AT%WIFICFG=\"SET\",\"TIMEOUT\",120,2000");
    wait_for("OK", 3000);
    send_at("AT%WIFICFG=\"SET\",\"SCANTABLE\",20,0");
    wait_for("OK", 3000);
    send_at("AT%WIFIEV=\"REGSCAN\",1");
    wait_for("OK", 3000);

    /* --- Passage en mode basse consommation piloté par DTR ---
     * DTR HIGH = fonctionnement normal, DTR LOW = veille
     * Le module reste enregistré sur le réseau dans les deux états.
     */
    send_at("AT+CFUN=5");
    wait_for("OK", 5000);
```

Cette séquence configure le module une seule fois au démarrage :

- Vérification de la liaison AT

- Activation du contexte de données LTE sur l'APN de l'opérateur 1NCE
- Synchronisation de l'horloge via NITZ (réseau LTE)
- Activation de l'eDRX pour réduire la consommation radio entre les cycles
- Configuration du scanner WiFi (canaux 1, 6, 11 ; timeout ; nombre de réseaux max)
- AT+CFUN=5 : active le mode de contrôle de veille par le signal DTR

---

13. main — Boucle principale

```c
    /* — Boucle principale : scan -> veille -> réveil -> répéter ——— */
    while (1) {
        wifi_scan_cycle();

        /* Affiche l'heure du prochain scan avant de dormir */
        print_next_scan_time();

        printk("\n--- Passage en veille (DTR LOW) - réveil dans %d s (%d min) ---\n",
               SLEEP_MS / 1000, SLEEP_MS / 60000);
        dtr_set(0);
        k_msleep(SLEEP_MS);

        printk("--- Réveil (DTR HIGH) ---\n");
        dtr_set(1);
        k_msleep(3000); /* laisse le temps au module de se réveiller complètement */
    }

    return 0;
}
```

La boucle infinie répète indéfiniment le même cycle :

1. **Scan WiFi** — détecte et affiche les réseaux environnants
2. **Affichage** de l'heure du prochain scan
3. **Veille** — le DTR passe à LOW, le module Telit entre en veille basse consommation ; le nRF dort 5 minutes avec k_msleep
4. **Réveil** — le DTR repasse a HIGH, le module se réveille ; 3 secondes d'attente lui laissent le temps de se réinitialiser avant le prochain cycle

Explication du fichier — nrf54l15dk_nrf54l10_cpuapp_ns.overlay

---

Dans Zephyr, le Device Tree est le fichier qui décrit le matériel de la carte. Le fichier .overlay permet de modifier ou compléter ce Device Tree de base sans toucher aux fichiers officiels de la carte. C'est ici qu'on déclare la configuration UART propre au projet.

---

1. Configuration des broches — bloc &pinctrl

```
&pinctrl {
    uart21_default: uart21_default {
        group1 {
            psels = <NRF_PSEL(UART_TX, 1, 9)>;
        };
        group2 {
            psels = <NRF_PSEL(UART_RX, 1, 10)>;
            bias-pull-up;
        };
    };
};
```

Ce bloc définit le **multiplexage des broches** (pinctrl = pin control) pour l'UART21, c'est-à-dire à quelles broches physiques du nRF54L15 sont connectés les signaux TX et RX.

- **group1 — TX** : le signal d'émission UART est assigné à la broche **P1.09** (NRF_PSEL(UART_TX, 1, 9) → port 1, pin 9)
- **group2 — RX** : le signal de réception est assigné à la broche **P1.10** (port 1, pin 10). La propriété bias-pull-up active la résistance de tirage interne vers le haut sur cette broche, ce qui est une pratique standard sur les lignes UART pour éviter des données parasites quand la ligne est au repos

uart21_default est le nom de cette configuration ; il sera référencé dans le bloc suivant.

2.Activation du périphérique UART — bloc &uart21

```
&uart21 {
    status = "okay";
    current-speed = <115200>;
    pinctrl-0 = <&uart21_default>;
    pinctrl-names = "default";
};
```

Ce bloc active et configure le périphérique UART21 du nRF54L15 :

- **status = "okay"** : active le périphérique (par défaut dans Zephyr, les périphériques sont désactivés ; il faut les activer explicitement)
- **current-speed = <115200>** : fixe la vitesse de communication à 115 200 bauds, ce qui correspond à la vitesse par défaut attendue par le module Telit ME310M1
- **pinctrl-0 = <&uart21_default>** : associe à cet UART la configuration de broches définie juste au-dessus

- **pinctrl-names = "default"** : indique que pinctrl-0 est la configuration active par défaut

[image ½ resultat terminal en haut du resultat final]

```
< %WIFIEVU: "REGSCAN",0,3,11
> AT%WIFICMD="GETRES"
< AT%WIFICMD="GETRES"
  [WIFI] %WIFICMD: "8E:78:48:DA:3A:71",-66,854,1,"",1
  [WIFI] %WIFICMD: "8A:78:48:DA:3A:71",-66,754,1,"INFPS-Etudiants",1
  [WIFI] %WIFICMD: "CC:2D:1B:F2:F1:B6",-82,923,11,"SFR_F1B0",11
  [WIFI] %WIFICMD: "2C:08:23:A5:E9:32",-88,840,1,"Livebox-034E",1
  [WIFI] %WIFICMD: "E4:5D:51:35:DC:B6",-74,1261,1,"SFR_DCB0",1
  [WIFI] %WIFICMD: "D0:6E:DE:B9:79:90",-84,1069,6,"Bbox-B9FA3940",6
  [WIFI] %WIFICMD: "3A:07:16:57:F6:54",-87,981,11,"Freebox-2A6FEA",11
  [WIFI] %WIFICMD: "A8:BA:25:85:CB:00",-88,1246,1,"PDAWLAN",1
  [WIFI] %WIFICMD: "32:CD:A7:BE:06:9A",-81,1137,6,"DIRECT-svM2070 Series",6
  [WIFI] %WIFICMD: "B8:B7:F1:11:8E:FA",-79,160,6,"Livebox-AJHAG",6
  [WIFI] %WIFICMD: "18:86:37:3B:1C:F0",-75,1305,1,"Livebox-1CF0",1
  [WIFI] %WIFICMD: "84:78:48:DA:3A:71",-64,758,1,"INFPS-Admin",1
  [WIFI] %WIFICMD: "38:C0:EA:7E:95:79",-67,804,1,"4MOD-Guest",1
  [WIFI] %WIFICMD: "58:1D:D8:4B:FE:18",-82,777,1,"Livebox-FE10",1
  [WIFI] %WIFICMD: "38:C0:EA:7E:95:78",-67,398,1,"4MOD",1
```

Je flashe le module et on arrive a bien voir tout les wifi environnant, l'implémentation sur micro controleur a bien fonctionné

GNSS

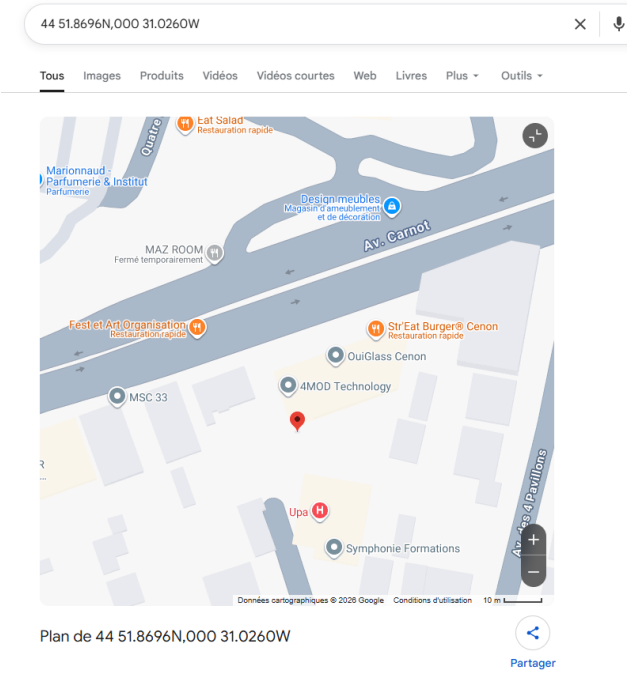
Mise en place du GNSS

```
OK
AT+CFUN=4
OK
AT$GPSP=1
OK
AT$GNSSNMEA=1,1
OK
```

Etant donné que le GNSS et le LTE ne sont pas compatible il faut se mettre en CFUN 4

Au bout d'une certaine période on arrive au fix (connexion et information donné par satellites qui nous donné une position

```
$GPRMC,090030.00,V,,,,,,,160426,,,N*70
$GPRMC,090031.00,V,,,,,,,160426,,,N*71
$GPRMC,090031.79,A,4451.8655,N,00031.0001,W,0.0,0.0,160426,,,A*47
$GPRMC,090032.00,A,4451.8655,N,00031.0001,W,0.0,0.0,160426,,,A*4A
$GPRMC,090033.00,A,4451.8655,N,00031.0001,W,0.0,0.0,160426,,,A*4B
$GPRMC,090034.00,A,4451.8677,N,00031.0148,W,0.0,0.0,160426,,,A*40
$GPRMC,090035.00,A,4451.8669,N,00031.0139,W,0.0,0.0,160426,,,A*48
$GPRMC,090036.00,A,4451.8676,N,00031.0135,W,0.0,0.0,160426,,,A*49
$GPRMC,090037.00,A,4451.8682,N,00031.0131,W,0.0,0.0,160426,,,A*47
```



La localisation est bonne, le GNSS est validé, il envoie une position par seconde.

Pour l'implémentation dans le code il faudrait :

-Desactiver edrx

-Desactiver cFUN5 au pr

-Dé-connexion du reseau obligatoire *« iGNSS engine can work if LTE is off, so for starting a iGNSS positioning session, a CFUN=4 is needed."*

Je vais donc l'ajouter au code de l'uC

[SCREEN AJOUT DU CODE]

Résultat :

```
> AT$GNSSNMEA=1,1
< AT$GNSSNMEA=1,1
< OK
  [GNSS] $GPRMC,135055.00,V,4451.8446,N,00030.9722,W,,,170426,,,N*52
  [GNSS] $GPRMC,135055.77,A,4451.8680,N,00030.9880,W,0.3,0.0,170426,,,A*46
[GNSS] Fix obtenu!
  [GNSS] $GPRMC,135056.00,A,4451.8680,N,00030.9880,W,0.3,0.0,170426,,,A*45
  [GNSS] $GPRMC,135057.00,A,4451.8564,N,00030.9947,W,0.3,0.0,170426,,,A*47
  [GNSS] $GPRMC,135058.00,A,4451.8539,N,00031.0193,W,0.4,0.0,170426,,,A*4E
  [GNSS] $GPRMC,135059.00,A,4451.8534,N,00031.0245,W,0.4,0.0,170426,,,A*4A
  [GNSS] $GPRMC,135100.00,A,4451.8540,N,00031.0222,W,0.0,0.0,170426,,,A*41
> AT$GNSSNMEA=0,0
< AT$GNSSNMEA=0,0
```

Deux minutes pour se fix, au moment ou le fix est done encore 5 coordonnées puis désactivation de l'envoie des données GNSS

Les deux peuvent dont fonctionner mais JAMAIS en même temps, avec le microcontrôleur on peut tout automatiser pour ça aille vite mais lorsque le GNSS est activé le LTE est forcément désactiver, ils ne fonctionnent pas en même temps.

Envoi des réponses sur serveur

Il y avait plusieurs possibilités de protocole, j'ai choisi le MQTT.

Pour cela je me suis créé un compte gratuitement sur HiveMQ Cloud, c'est bien assez pour de simples tests actuellement. HiveMQ propose une interface web permettant de visualiser en temps réel les messages reçus sur un topic donné.

Le module Telit ME310M1 se connecte au broker HiveMQ Cloud sur le port 8883, en utilisant les commandes AT dédiées (AT#MQCONN, AT#MQPUBS). Après chaque cycle de scan WiFi, les résultats sont publiés sur le topic tracker/wifi avec le flag *retain* activé, ce qui permet de toujours consulter le dernier message même en se connectant après la publication.

```
0  Topic: tracker/wifi  QoS: 0
4MOD-Guest -52dBm ch1 38:C0:EA:7E:95:79 // Livebox-FE10 -60dBm ch11 58:1D:D8:4B:FE:18 // 4MOD
-48dBm ch1 38:C0:EA:7E:95:78 // TP-LINK_BB58 -66dBm ch1 F4:F2:6D:D5:BB:5
1  Topic: tracker/wifi  QoS: 0
Livebox-D4A0 -67dBm ch11 78:C2:13:D7:D4:A0 // SFR_47FF_5GHZ -80dBm ch11 6A:57:3E:64:48:06 //
WIFI CLIENT SGS -83dBm ch1 A8:BA:25:85:CB:02 // 4MOD-Guest -53dBm ch1 38:C0:EA:7E:95:79 // Li
vebox-FE10 -58dBm ch11 58:1D:D8:4B:FE:18 // 4MOD -50dBm ch1 38:C0:EA:7E:95:78 // TP-LINK_BB58
-69dBm ch1 F4:F2:6D:D5:BB:58 // AP00014428 -87dBm ch1 C2:90:78:C2:4D:0
2  Topic: tracker/wifi  QoS: 0
(hidden) -73dBm ch11 18:31:BF:9F:50:E0 // Livebox-D4A0 -68dBm ch11 78:C2:13:D7:D4:A0 // WIFI
CLIENT SGS -84dBm ch1 A8:BA:25:85:CB:02 // PDAWLAN -82dBm ch1 A8:BA:25:85:CB:00 // EQWLAN -82
dBm ch1 A8:BA:25:85:CB:01 // 4MOD-Guest -52dBm ch1 38:C0:EA:7E:95:79 // 4MOD -49dBm ch1 38:C
0:EA:7E:95:78 // AP00014428 -86dBm ch1 C2:90:78:C2:4D:0
```

```
< AT%WIFICMD="GETRES"
< %WIFICMD: "78:C2:13:D7:D4:A0",-60,706,6,"Livebox-D4A0",6
< %WIFICMD: "A8:BA:25:85:CB:02",-82,1283,1,"WIFI CLIENT SGS",1
< %WIFICMD: "A8:BA:25:85:CB:01",-82,772,1,"EQWLAN",1
< %WIFICMD: "38:C0:EA:7E:95:79",-53,1252,1,"4MOD-Guest",1
< %WIFICMD: "58:1D:D8:4B:FE:18",-51,752,1,"Livebox-FE10",1
< %WIFICMD: "38:C0:EA:7E:95:78",-50,1256,1,"4MOD",1
< %WIFICMD: "F4:F2:6D:D5:BB:58",-70,1183,1,"TP-LINK_BB58",2
[WIFI] OK
> AT%WIFICMD="CLEARRES"
< AT%WIFICMD="GETRES"
< %WIFICMD: "18:31:BF:9F:50:E0",-71,936,11,"",13
< %WIFICMD: "A8:BA:25:85:CB:02",-82,766,1,"WIFI CLIENT SGS",1
< %WIFICMD: "A8:BA:25:85:CB:00",-79,767,1,"PDAWLAN",1
< %WIFICMD: "A8:BA:25:85:CB:01",-81,358,1,"EQWLAN",1
< %WIFICMD: "38:C0:EA:7E:95:79",-53,328,1,"4MOD-Guest",1
< %WIFICMD: "38:C0:EA:7E:95:78",-49,331,1,"4MOD",1
< %WIFICMD: "F4:F2:6D:D5:BB:58",-69,769,1,"TP-LINK_BB58",2
[WIFI] OK
< AT%WIFICMD="GETRES"
< %WIFICMD: "78:C2:13:D7:D4:A0",-57,166,6,"Livebox-D4A0",6
< %WIFICMD: "6A:57:3E:64:48:06",-77,916,11,"SFR_47FF_5GHZ",11
< %WIFICMD: "A8:BA:25:85:CB:02",-85,1246,1,"WIFI CLIENT SGS",1
< %WIFICMD: "A8:BA:25:85:CB:00",-79,834,1,"PDAWLAN",1
< %WIFICMD: "A8:BA:25:85:CB:01",-85,833,1,"EQWLAN",1
< %WIFICMD: "38:C0:EA:7E:95:79",-53,806,1,"4MOD-Guest",1
< %WIFICMD: "38:C0:EA:7E:95:78",-49,1219,1,"4MOD",1
< %WIFICMD: "F4:F2:6D:D5:BB:58",-65,1249,1,"TP-LINK_BB58",2
< %WIFICMD: "C2:90:78:C2:4D:08",-86,1270,1,"AP00014428",1
[WIFI] OK
```

Comme on peut le voir sur la capture ci-dessus, chaque message reçu sur HiveMQ contient la liste des points d'accès WiFi détectés, avec pour chacun : le SSID, le niveau de signal en dBm, le canal radio et l'adresse MAC. Ces informations sont séparées par // pour faciliter la lecture. On retrouve par exemple le réseau 4MOD à -49 dBm sur le canal 1, ou encore Livebox-FE10 à -52 dBm sur le canal 6.

Ces données constituent la base nécessaire pour une localisation par empreinte WiFi (*WiFi fingerprinting*).

Je vais ensuite faire la même chose avec le même principe mais cette fois ci pour le GNSS, et ensuite essayer de trouver le temps minimal que je peux avoir entre deux envoie de données.

Implémentation GNSS dans le microcontrôleur

Maintenant que le GNSS fonctionne en commandes AT directes et que le WiFi est déjà intégré dans le code du microcontrôleur, l'étape suivante est d'automatiser la session GNSS depuis le nRF54L15.

La contrainte principale est connue depuis les tests manuels : le LTE et le GNSS ne peuvent pas fonctionner en même temps. Le microcontrôleur doit donc gérer lui-même la bascule CFUN=4 (LTE off → GNSS on) puis le retour CFUN=1 (LTE on → GNSS off) après chaque acquisition.

Détection d'un fix valide

La première fonction ajoutée est is_gnss_fix(), qui filtre les trames NMEA reçues sur l'UART pour ne retenir que celles contenant une position réelle.

```c
static int is_gnss_fix(const char *line)
{
    if (strncmp(line, "$GPRMC,", 7) != 0 &&
        strncmp(line, "$GNRMC,", 7) != 0)
        return 0;
    const char *comma = strchr(line + 7, ',');
    return comma && *(comma + 1) == 'A';
}
```

Le module envoie en continu des trames $GPRMC (ou $GNRMC). Elles contiennent un champ statut : V signifie que le récepteur n'a pas encore de fix, A signifie que la position est valide. La fonction cherche ce champ et retourne 1 uniquement si le statut est A.

Conversion des coordonnées NMEA en degrés décimaux

Les trames NMEA n'expriment pas les coordonnées en degrés décimaux. Elles utilisent le format DDMM.MMMM (degrés + minutes décimales), qu'il faut convertir. La formule est : degrés décimaux = DD + MM.MMMM / 60.

La conversion a donc été implémentée

```c
    int64_t scale = 1;
    for (int i = 0; i < lat_frac_digits; i++) scale *= 10;
    /* lat en nanodegrés = (lat_dd + (lat_mm + lat_frac/scale) / 60) * 1e9  */
    int64_t lat_nano = (int64_t)lat_dd * 1000000000LL
                     + ((int64_t)lat_mm * 1000000000LL
                     + (int64_t)lat_frac * (1000000000LL / scale)) / 60;
    if (*f4 == 'S') lat_nano = -lat_nano;

    scale = 1;
    for (int i = 0; i < lon_frac_digits; i++) scale *= 10;
    int64_t lon_nano = (int64_t)lon_ddd * 1000000000LL
                     + ((int64_t)lon_mm * 1000000000LL
                     + (int64_t)lon_frac * (1000000000LL / scale)) / 60;
    if (*f6 == 'W') lon_nano = -lon_nano;

    /* --- formatage : on affiche 7 décimales (précision ~1cm) --- */
    int lat_neg = lat_nano < 0; if (lat_neg) lat_nano = -lat_nano;
    int lon_neg = lon_nano < 0; if (lon_neg) lon_nano = -lon_nano;

    int lat_int = (int)(lat_nano / 1000000000LL);
    int lat_dec = (int)(lat_nano % 1000000000LL / 100); /* 7 décimales */
    int lon_int = (int)(lon_nano / 1000000000LL);
    int lon_dec = (int)(lon_nano % 1000000000LL / 100);

    snprintf(gnss_payload, sizeof(gnss_payload),
        "%s%d.%07d,%s%d.%07d",
        lat_neg ? "-" : "", lat_int, lat_dec,
        lon_neg ? "-" : "", lon_int, lon_dec);
}
```

https://stackoverflow.com/questions/14421651/conversion-of-gps-coordinate-units-using-c

https://forum.arduino.cc/t/gps-coordinate-conversion-help/109169/2

Le résultat est formaté avec 7 décimales via %d.%07d, ce qui donne par exemple 44.8655000,-0.0310010.

Cycle de tracking GNSS

La fonction gnss_tracking_cycle() gère l'ensemble de la session GNSS de manière autonome.

```c
static void gnss_tracking_cycle(void)
{
    char line[BUF_SIZE];
    int  publish_count = 0;

    gnss_payload[0] = '\0';
    printk("=== Tracking GNSS : %d envois x %d trames ===\n",
           GNSS_SEND_COUNT, GNSS_FRAMES_PER_SEND);

    while (publish_count < GNSS_SEND_COUNT) {
        int valid_count = 0;
        int timeout_ms = (publish_count == 0) ? GNSS_TIMEOUT_MS
                                              : GNSS_REFIX_TIMEOUT_MS;

        /* --- Passe en mode GNSS (radio off) --- */
        send_at("AT+CFUN=4");
        wait_for("OK", 10000);
        k_msleep(1000); /* stabilisation avant GPSP */

        send_at("AT$GPSP=0"); /* arrêt préventif si GNSS déjà actif */
        wait_for("OK", 3000);

        send_at("AT$GPSP=1");
        if (wait_for("OK", 10000) != 0) {
            printk("[GNSS] Echec demarrage GNSS\n");
            gnss_restore_lte();
            return;
        }

        send_at("AT$GNSSNMEA=1,1");
        wait_for("OK", 3000);
```

```c
        /* --- Accumule GNSS_FRAMES_PER_SEND trames valides --- */
        {
            int64_t end = k_uptime_get() + timeout_ms;
            while (k_uptime_get() < end) {
                if (k_msgq_get(&telit_msgq, line, K_MSEC(200)) == 0) {
                    print_line(line);
                    if (is_gnss_fix(line)) {
                        gnss_build_payload(line);
                        valid_count++;
                        printk("[GNSS] Trame %d/%d : %s\n",
                               valid_count,
                               GNSS_FRAMES_PER_SEND,
                               gnss_payload);
                        if (valid_count >= GNSS_FRAMES_PER_SEND)
                            break;
                    }
                }
            }
        }

        /* --- Arrêt GNSS --- */
        send_at("AT$GNSSNMEA=0,0");
        wait_for("OK", 3000);
        send_at("AT$GPSP=0");
        wait_for("OK", 5000);
```

```
        /* --- Retour LTE --- */
        gnss_restore_lte();

        if (valid_count < GNSS_FRAMES_PER_SEND) {
            printk("[GNSS] Seulement %d/%d trames, fin tracking\n",
                   valid_count, GNSS_FRAMES_PER_SEND);
            return;
        }

        /* --- Envoi MQTT --- */
        if (mqtt_connect() == 0) {
            mqtt_publish_gnss();
            mqtt_disconnect();
            publish_count++;
            printk("[GNSS] Envoi %d/%d OK\n", publish_count, GNSS_SEND_COUNT);
        }
    }

    printk("[GNSS] Tracking termine : %d positions envoyees\n", GNSS_SEND_COUNT);
}
```

Le principe : répéter 5 fois la séquence suivante :

Passer en CFUN=4 (mode avion)

Démarrer le GNSS (AT$GPSP=1 + AT$GNSSNMEA=1,1)

Attendre 5 trames valides (statut A et pas V)

Arrêter le GNSS (AT$GNSSNMEA=0,0 + AT$GPSP=0)

Revenir en LTE et publier la position sur MQTT

Le premier fix a 2 minutes pour réussir (si dehors ou dans la salle de pause max 30 secondes pour se connecter la plupart du temps), (GNSS_TIMEOUT_MS), les suivants de 1 minute (GNSS_REFIX_TIMEOUT_MS) car il y a la mémoire du premier fix

Retour LTE après une session GNSS

Le retour en LTE n'est pas immédiat. CFUN=4 désactive aussi le contexte PDP (la session de données réseau), il faut le réactiver avant toute tentative MQTT. La fonction gnss_restore_lte() gère cette séquence :

```c
/* — Retour LTE après une session GNSS (CFUN=4 → CFUN=1 → LTE → CFUN=5) — */
static void gnss_restore_lte(void)
{
    send_at("AT+CFUN=1");
    wait_for("OK", 10000);
    wait_lte_registered(60000);

    /* CFUN=4 désactive le contexte PDP — le ré-activer si besoin. */
    send_at("AT#SGACT?");
    if (wait_for("#SGACT: 1,1", 3000) != 0) {
        send_at("AT#SGACT=1,1");
        wait_for("#SGACT:", 30000);
        wait_for("OK", 5000);
    } else {
        drain_queue(200);
    }

    /* Si CFUN=4 a effacé la config MQTT (broker vide), refaire mqtt_init.
     * Si la config est déjà là, ne pas toucher : MQCONN marchera direct
     * et tenter de reconfigurer SSL casserait la liaison existante. */
    {
        char cfg_line[BUF_SIZE];
        int  broker_set = 0;

        send_at("AT#MQCFG?");
        int64_t t = k_uptime_get() + 3000;
        while (k_uptime_get() < t) {
            if (k_msgq_get(&telit_msgq, cfg_line, K_MSEC(200)) == 0) {
                print_line(cfg_line);
                if (strstr(cfg_line, "#MQCFG: 1,") &&
                    !strstr(cfg_line, "1,\"\""))
                    broker_set = 1;
                if (strstr(cfg_line, "OK")) break;
            }
        }
        if (!broker_set)
            mqtt_init();
    }

    send_at("AT+CFUN=5");
    wait_for("OK", 5000);
}
```

Séquence appliquée à chaque retour :

AT+CFUN=1 > retour radio LTE

Attente réenregistrement réseau (AT+CEREG? en boucle)

Vérification AT#SGACT? : si le contexte PDP est tombé, réactivation AT#SGACT=1,1

Vérification AT#MQCFG? si la configuration MQTT a été effacée par le CFUN=4, réinitialisation complète

AT+CFUN=5 retour en mode veille eDRX piloté par DTR

Initialisation MQTT/TLS — connexion HiveMQ

```c
/* 1. Déconnecter et désactiver MQTT pour libérer toute liaison SSL */
send_at("AT#MQDISC=1");
wait_for("OK", 3000);
send_at("AT#MQEN=1,0");
wait_for("OK", 3000);

/* 2. Activer SSL, le configurer, puis le DÉSACTIVER avant MQCFG.
 *    MQCFG échoue avec "SSL already activated" si le socket est actif
 *    quand il tente de le lier en interne. On configure pendant qu'il
 *    est actif (requis par la doc), puis on l'éteint avant MQCFG. */
send_at("AT#SSLEN=1,1");
wait_for("OK", 3000);
send_at("AT#SSLCFG=1,1,0,90,100,50");
wait_for("OK", 3000);
send_at("AT#SSLSECCFG=1,0,0");
wait_for("OK", 3000);
send_at("AT#SSLSECCFG2=1,3,1");
wait_for("OK", 3000);
send_at("AT#SSLEN=1,0");   /* ← désactivé : MQCFG peut lier sans conflit */
wait_for("OK", 3000);

/* 3. Activer MQTT (état frais, SSL non encore lié) */
send_at("AT#MQEN=1,1");
if (wait_for("OK", 5000) != 0) {
    printk("[MQTT] Echec MQEN\n");
    return -1;
}

/* 4. Configurer MQTT+SSL (socket SSL désactivé → pas de conflit) */
{
    char cfg_cmd[100];
    snprintf(cfg_cmd, sizeof(cfg_cmd),
        "AT#MQCFG=1,\"%s\",%d,1,1,1", MQTT_BROKER, MQTT_PORT);
    send_at(cfg_cmd);
}
if (wait_for("OK", 5000) != 0) {
    printk("[MQTT] Echec MQCFG\n");
    return -1;
}

/* 5. Réactiver SSL pour l'utilisation réelle */
send_at("AT#SSLEN=1,1");
wait_for("OK", 3000);
```

La séquence (découverte par analyse de la documentation AT) :

MQDISC=1 > MQEN=1,0 : déconnecter et désactiver toute session existante

SSLEN=1,1 > SSLCFG > SSLSECCFG > SSLSECCFG2 : configurer SSL pendant qu'il est actif

SSLEN=1,0 : désactiver SSL AVANT MQCFG (requis par le module)

MQEN=1,1 : activer MQTT

MQCFG=1,"broker",8883,... : lier broker + SSL (SSL doit être off à ce moment)

SSLEN=1,1 : réactiver SSL

Si cette séquence n'est pas respectée (notamment le SSLEN=1,0 avant MQCFG), le module répond connection failed à la commande MQCFG.

Pour la connexion et publication, le firmware se connecte (AT#MQCONN), publie, puis se déconnecte (AT#MQDISC) à chaque cycle. Une connexion permanente ne peut pas être maintenue pendant les phases de veille eDRX ou wifi scan.

```c
/* ── Connexion MQTT — appelée à chaque cycle (config déjà en place) ─── */
static int mqtt_connect(void)
{
    printk("=== Connexion MQTT -> %s:%d ===\n", MQTT_BROKER, MQTT_PORT);

    /* Remet le client en état "init" si la connexion précédente est morte */
    send_at("AT#MQDISC=1");
    wait_for("OK", 3000);

    send_at("AT#MQCONN=1,\"" MQTT_CLIENT_ID "\",\"" MQTT_USER "\",\"" MQTT_PASS "\"");
    if (wait_for("OK", 30000) != 0) {
        printk("[MQTT] Echec MQCONN\n");
        return -1;
    }

    printk("[MQTT] Connecte\n");
    return 0;
}

/* ── Publication MQTT des résultats WiFi ──────────────────────────── */
static void mqtt_publish_wifi(void)
{
    char cmd[MQTT_BUF_SIZE + 80];
    const char *payload = mqtt_payload[0] ? mqtt_payload : "no_results";

    /* AT#MQPUBS=<inst>,<topic>,<retain>,<qos>,<message> */
    snprintf(cmd, sizeof(cmd),
         "AT#MQPUBS=1,\"" MQTT_TOPIC_WIFI "\",1,0,\"%s\"", payload); /* retain=1 */
    send_at(cmd);

    if (wait_for("OK", 10000) != 0) {
        printk("[MQTT] Echec publication\n");
    } else {
        printk("[MQTT] Publie sur " MQTT_TOPIC_WIFI "\n");
    }
}
```

```c
/* ── Publication MQTT des données GNSS ──────────────────────────── */
static void mqtt_publish_gnss(void)
{
    char cmd[128];

    snprintf(cmd, sizeof(cmd),
             "AT#MQPUBS=1,\"" MQTT_TOPIC_GNSS "\",1,0,\"%s\"", gnss_payload);
    send_at(cmd);

    if (wait_for("OK", 10000) != 0) {
        printk("[MQTT] Echec publication GNSS\n");
    } else {
        printk("[MQTT] Publie sur " MQTT_TOPIC_GNSS " : %s\n", gnss_payload);
    }
}
```

Résultats GNSS + WiFi Scan sur HiveMQ

0 Topic: tracker/gnss QoS: 0

44.8641650,-0.5167666

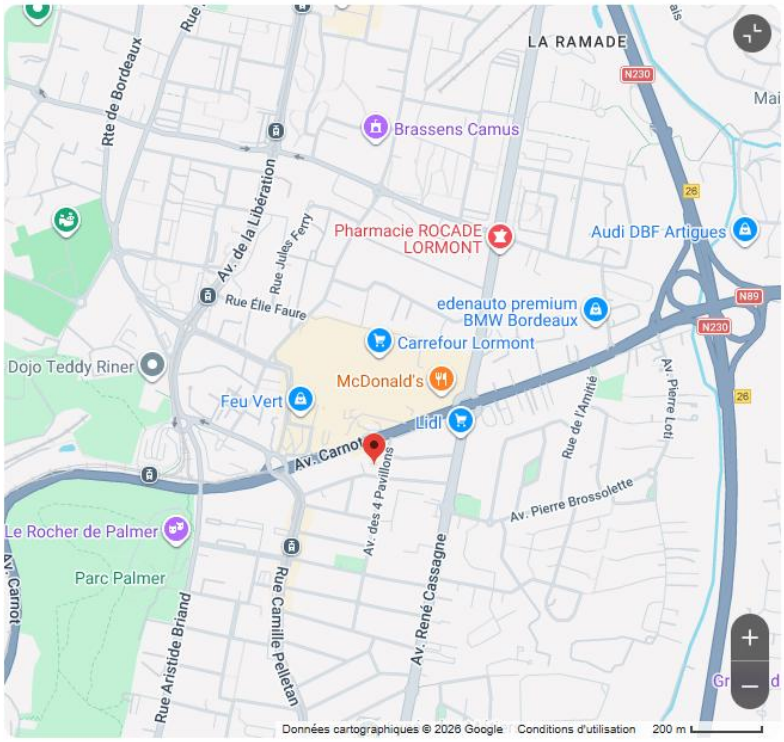
1 Topic: tracker/wifi QoS: 0

(hidden) -82dBm ch11 18:31:BF:9F:50:E0 // (hidden) -70dBm ch1 8E:78:48:DA:3A:71 // INFPS-Etudiants -71dBm ch1 8A:78:48:DA:3A:71 // SFR_F1B0 -75dBm ch1 CC:2D:1B:F2:F1:B6 // Livebox-034E -79dBm ch1 2C:08:23:A5:E9:32 // SFR_DCB0 -73dBm ch6 E4:5D:51:35:DC:B6 // Livebox-D4A0 -80dBm ch11 78:C2:13:D7:D4:A0 // Bbox-B9FA3940 -79dBm ch6 D0:6E:DE:B9:79:90 // SFR_47FF_5GHZ -80dBm ch11 6A:57:3E:64:48:06 // PDAWLAN -82dBm ch1 A8:BA:25:85:CB:0

10	Topic: tracker/gnss	QoS: 0
	44.8641700,-0.5166700	
11	Topic: tracker/gnss	QoS: 0
	44.8642650,-0.5167633	
12	Topic: tracker/gnss	QoS: 0
	44.8643766,-0.5167116	
13	Topic: tracker/gnss	QoS: 0
	44.8642800,-0.5163383	
14	Topic: tracker/gnss	QoS: 0
	44.8641650,-0.5167666	

```
33  Topic: tracker/wifi  QoS: 0
    SFR_629F -87dBm ch1 B0:B3:69:F0:62:A1 // INFPS-Etudiants -75dBm ch1 8A:78:48:DA:3A:71 // S
    FR_F1B0 -75dBm ch11 CC:2D:1B:F2:F1:B6 // SFR_DCB0 -79dBm ch11 E4:5D:51:35:DC:B6 // SFR_47F
    F_5GHZ -78dBm ch11 6A:57:3E:64:48:06 // WIFI CLIENT SGS -79dBm ch1 A8:BA:25:85:CB:02 // DI
    RECT-svM2070 Series -84dBm ch6 32:CD:A7:BE:06:9A // EQWLAN -77dBm ch1 A8:BA:25:85:CB:01 //
    4MOD-Guest -50dBm ch1 38:C0:EA:7E:95:79 // INFPS-Admin -76dBm ch1 84:78:48:DA:3A:7
34  Topic: tracker/gnss  QoS: 0
    44.8642883,-0.5168583
35  Topic: tracker/gnss  QoS: 0
    44.8644950,-0.5170083
36  Topic: tracker/gnss  QoS: 0
    44.8644366,-0.5170800
37  Topic: tracker/gnss  QoS: 0
    44.8643300,-0.5170966
```

Chaque message publié sur tracker/gnss contient la position au format lat,lon avec 7 décimales, par exemple : 44.8641766,-0.5167116



Plan de 44.8641766,-0.5167116

Partager

Le flag retain=1 est activé sur les deux topics (tracker/wifi et tracker/gnss), ce qui permet de toujours consulter la dernière position connue sur HiveMQ même en se connectant après la publication.

Code final — boucle principale complète

```
/* — Boucle principale : scan -> MQTT -> veille -> réveil -> répéter — */
while (1) {
    wifi_scan_cycle();
    gnss_tracking_cycle();

    if (mqtt_connect() == 0) {
        mqtt_publish_wifi();
        mqtt_disconnect();
    }

    print_next_scan_time();

    printk("\n--- Passage en veille (DTR LOW) - réveil dans %d s (%d min) ---\n",
           SLEEP_MS / 1000, SLEEP_MS / 60000);
    dtr_set(0);
    k_msleep(SLEEP_MS);

    printk("--- Réveil (DTR HIGH) ---\n");
    dtr_set(1);
    k_msleep(3000); /* laisse le temps au module de se réveiller */
    wifi_init();    /* re-initialise le WiFi après la veille DTR */
}

return 0;
}
```

La boucle principale enchaîne désormais :

Scan WiFi > capture SSID / RSSI / canal / MAC

Tracking GNSS — 5 × (5 trames valides → publication tracker/gnss)

Publication WiFi > résultats du scan sur tracker/wifi

Veille 5 minutes > DTR LOW, module en basse consommation

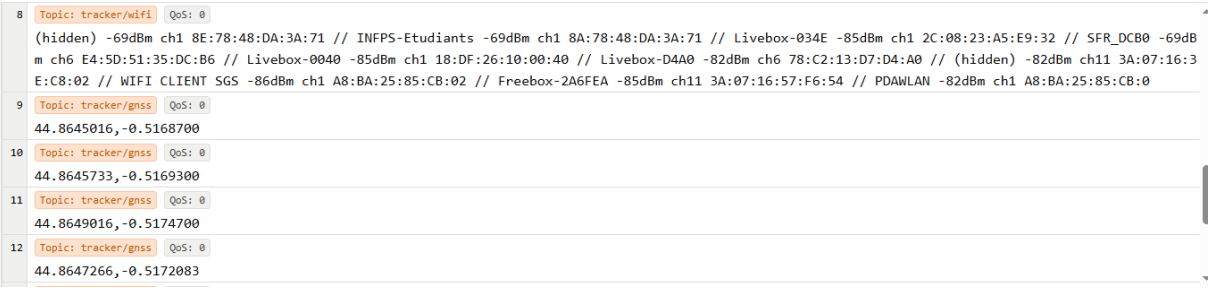
Réveil > DTR HIGH, ré-initialisation WiFi, recommencer

Le WiFi et le GNSS fonctionnent dans le même firmware mais pas simultanément : le WiFi s'exécute en CFUN=5 (DTR HIGH), le GNSS impose un passage en CFUN=4 (LTE off) pour chaque acquisition, CFUN qu'il faut également changer pour pouvoir envoyer les données au serveur HiveMQ

le système fonctionne et la communication avec le serveur se fait bien (scan wifi, acquisition GNSS et envoi des positions sur hiveMQ avec MQTT la logique que j'ai mise en place en premier lieu pour juste réussir la communication est :

Toutes les 5 trames GNSS valides -> envoi de la position répété 5 fois par cycle et après veille de 5 minutes

Détermination de la cadence minimale



Optimisation de la cadence d'envoie des données GNSS

Une fois les fonctions GNSS et MQTT validées individuellement, il a fallu déterminer la cadence à laquelle le tracker peut enchaîner les cycles de collecte sans déstabiliser la SIM 1NCE. Le cycle est le suivant : passage en mode GNSS (AT+CFUN=4) > fix > retour LTE >envoi MQTT > attente. C'est la durée minimale entre deux cycles que je vais tester et baisser au fur et a mesure

Un mode de test dédié a été implémenté (gnss_timing_test) en conservant strictement la même base de code existante. À chaque itération, la fonction mesurait le temps total du cycle (fix + reconnexion + MQTT) et attendait le reliquat jusqu'à atteindre un minimum configurable via la constante GNSS_CYCLE_MIN_MS. Les logs de sortie affichaient systématiquement le temps de fix, le temps total du cycle, et un compteur de réussite cumulé.

!SCREEN ! montrer screen avec stats d'envoie direct dans le terminal avec ms et tout

Les tests ont montré qu'avec une valeur de GNSS_CYCLE_MIN_MS = 15000 ms (15 secondes), des instabilités réseau apparaissaient sur la SIM 1NCE . En mettant à 20 secondes ça restait stable sur toute la durée du test.

Architecture du cycle final

avec ce résultat, voici les paramètres de comportement qui sont regroupés en tête de fichier :

```
#define TELIT_UART_NODE  DT_NODELABEL(uart21)
#define BUF_SIZE         256
#define DTR_PIN          11      /* P1.11 -> EVB GPIO01 -> module C108/DTR */
#define NB_SCANS         3       /* nombre de scans WiFi par cycle         */
#define SLEEP_MS         1200000 /* durée de veille : 20 minutes           */
#define SLEEP_MIN        (SLEEP_MS / 60000)
#define GNSS_TIMEOUT_MS      120000  /* 2 min max pour le premier fix GNSS     */
#define GNSS_REFIX_TIMEOUT_MS 60000  /* 1 min max pour un re-fix après arrêt   */
#define CYCLE_COUNT               5  /* cycles wifi+gnss par session           */
#define GNSS_CYCLE_MIN_MS     20000  /* délai min entre deux cycles            */
```

La fonction gnss_one_fix() encapsule l'intégralité du cycle GNSS :

-passage en CFUN=4 démarrage du récepteur (AT$GPSP=1), écoute du flux jusqu'à obtenir une trame $GPRMC/$GNRMC avec statut A, puis arrêt du récepteur et appel à gnss_restore_lte() pour revenir en LTE. Un flag static distingue le premier fix (timeout 2 min pour le coldfilx) des fix suivants (timeout 1 min) une fois qu'on arrive à avoir un hotfix.

```
[GNSS] $GPRMC,141354.00,V,4451.8633,N,00031.0051,W,,,230426,,,N*5F
[GNSS] $GPRMC,141355.00,V,4451.8633,N,00031.0051,W,,,230426,,,N*5E
[GNSS] $GPRMC,141356.00,V,4451.8633,N,00031.0051,W,,,230426,,,N*5D
[GNSS] $GPRMC,141357.00,V,4451.8633,N,00031.0051,W,,,230426,,,N*5C
[GNSS] $GPRMC,141358.00,V,4451.8633,N,00031.0051,W,,,230426,,,N*53
[GNSS] $GPRMC,141359.00,V,4451.8633,N,00031.0051,W,,,230426,,,N*52
[GNSS] $GPRMC,141400.00,V,4451.8633,N,00031.0051,W,,,230426,,,N*59
[GNSS] $GPRMC,141401.89,A,4451.8716,N,00031.0140,W,0.0,0.0,230426,,,A*46
[GNSS] Fix en 37785 ms : 44.8645266,-0.5169000
```

Cold fix pour le premier scan GNSS (37sec)puis hotfix pour les autres meme après sommeil :

```
[WIFI] OK
[GNSS] $GPRMC,143907.68,A,4451.8763,N,00031.0220,W,0.0,284.8,230426,,,A*41
[GNSS] Fix en 997 ms : 44.8646050,-0.5170333
```

```
[WIFI] OK
[GNSS] $GPRMC,141454.00,V,4451.8585,N,00031.0225,W,,,230426,,,N*57
[GNSS] $GPRMC,141455.00,V,4451.8585,N,00031.0225,W,,,230426,,,N*56
[GNSS] $GPRMC,141455.76,A,4451.8293,N,00031.0225,W,0.0,180.1,230426,,,A*47
[GNSS] Fix en 4074 ms : 44.8638216,-0.5170416
```

Hotfix (4sec et 0.9sec) pour chacune des nouvelles captures GNSS

```c
/* ── Capture un seul fix GNSS puis restaure le LTE ──────────────────── */
static int gnss_one_fix(void)
{
    char    line[BUF_SIZE];
    int     got_fix = 0;
    static  int first = 1;
    int     timeout = first ? GNSS_TIMEOUT_MS : GNSS_REFIX_TIMEOUT_MS;

    send_at("AT+CFUN=4");
    wait_for("OK", 10000);
    k_msleep(1000);

    send_at("AT$GPSP=0");
    wait_for("OK", 3000);

    send_at("AT$GPSP=1");
    if (wait_for("OK", 10000) == 0) {
        send_at("AT$GNSSNMEA=1,1");
        wait_for("OK", 3000);

        int64_t fix_start    = k_uptime_get();
        int64_t fix_deadline = fix_start + timeout;
```

```
        while (!got_fix && k_uptime_get() < fix_deadline) {
            if (k_msgq_get(&telit_msgq, line, K_MSEC(200)) == 0) {
                print_line(line);
                if (is_gnss_fix(line)) {
                    gnss_build_payload(line);
                    printk("[GNSS] Fix en %lld ms : %s\n",
                           (long long)(k_uptime_get() - fix_start),
                           gnss_payload);
                    got_fix = 1;
                    first   = 0;
                }
            }
        }
        send_at("AT$GNSSNMEA=0,0");
        wait_for("OK", 3000);
    } else {
        printk("[GNSS] Echec GPSP=1\n");
    }

    send_at("AT$GPSP=0");
    wait_for("OK", 5000);

    gnss_restore_lte();

    if (!got_fix) printk("[GNSS] Pas de fix\n");
    return got_fix;
}
```

La boucle principale fait ça:

Un seul scan WiFi (wifi_scan_cycle()) suivi d'un envoi MQTT immédiat

5 cycles GNSS (for i in 0..CYCLE_COUNT) : appel à gnss_one_fix(), publication GNSS si fix obtenu, attente jusqua 20 secondes.

Veille 20 minutes (j'ai également fais des tests avec 10 et 5, tous fonctionnels, j'ai augmenté pour compenser l'énorme changement de conso) : DTR LOW > k_msleep(SLEEP_MS) > DTR HIGH > réinitialisation WiFi.

```c
/* — Boucle principale : 1 scan WiFi puis CYCLE_COUNT cycles gnss+mqtt — */
while (1) {
    int ok_count = 0;

    wifi_scan_cycle();

    if (mqtt_connect() == 0) {
        mqtt_publish_wifi();
        mqtt_disconnect();
    }

    for (int i = 0; i < CYCLE_COUNT; i++) {
        int64_t t_start = k_uptime_get();
        printk("\n=== Cycle %d/%d ===\n", i + 1, CYCLE_COUNT);

        if (gnss_one_fix() && mqtt_connect() == 0) {
            mqtt_publish_gnss();
            mqtt_disconnect();
            ok_count++;
        }

        int64_t leftover = GNSS_CYCLE_MIN_MS - (k_uptime_get() - t_start);
        if (leftover > 0) k_msleep((int32_t)leftover);
    }

    print_next_scan_time();
    printk("=== Session : %d/%d OK — veille %d s ===\n",
           ok_count, CYCLE_COUNT, SLEEP_MS / 1000);

    dtr_set(0);
    k_msleep(SLEEP_MS);
    printk("--- Réveil ---\n");
    dtr_set(1);
    k_msleep(3000);
    wifi_init();
}
```

A la fin on a donc : 1 point WiFi + jusqu'à 5 points GNSS envoyés sur HiveMQ en moins de 2 minutes, après sleep de 20 minutes économisant la batterie.

```
8  Topic: tracker/wifi  QoS: 0
(hidden) -69dBm ch1 8E:78:48:DA:3A:71 // INFPS-Etudiants -69dBm ch1 8A:78:48:DA:3A:71 // Livebox-034E -85dBm ch1 2C:08:23:A5:E9:32 // SFR_DCB0 -69dB
m ch6 E4:5D:51:35:DC:B6 // Livebox-0040 -85dBm ch1 18:DF:26:10:00:40 // Livebox-D4A0 -82dBm ch6 78:C2:13:D7:D4:A0 // (hidden) -82dBm ch11 3A:07:16:3
E:C8:02 // WIFI CLIENT SGS -86dBm ch1 A8:BA:25:85:CB:02 // Freebox-2A6FEA -85dBm ch11 3A:07:16:57:F6:54 // PDAWLAN -82dBm ch1 A8:BA:25:85:CB:0
9  Topic: tracker/gnss  QoS: 0
44.8645016,-0.5168700
10 Topic: tracker/gnss  QoS: 0
44.8645733,-0.5169300
11 Topic: tracker/gnss  QoS: 0
44.8649016,-0.5174700
12 Topic: tracker/gnss  QoS: 0
44.8647266,-0.5172083
```

Limites finales de l'objet

Etant donné que ne nous sommes pas encore à la conception finale de l’objet mes recherches vont surtout servir a indiqués quels sont les limites de l’objet et les différents protocoles, modes etc.. Je vais recenser ça ici :

Consommation faible :

La consommation de « bord » du module, est de ~8mA

Aujourd’hui 23/04/2026 :

Réessayer envoie données GNSS plus fréquemment ✓

Vérifier si le protocole est le plus optimisé pour wifiscan + gnss + envoi serveur = conso d’énergie

eDRX Cycle Length

eDRX Cycle length controls the interval at which a modem turns on its receiver to check for incoming downlink data. eDRX Cycle lengths are specified using 4-bit binary string with values mapped directly to specific lengths for each of LTE-M and NB-IoT.

0000: 5.12 seconds
0001: 10.24 seconds
0010: 20.48 seconds
0011: 40.96 seconds
0100: 61.44 seconds
0101: 81.92 seconds
0110: 102.4 seconds
0111: 122.88 seconds
1000: 143.36 seconds
1001: 163.84 seconds
1010: 327.68 seconds
1011: 655.36 seconds
1100: 1310.72 seconds
1101: 2621.44 seconds

https://developers.soracom.io/en/docs/air/edrx-psm-timers/

Courriel support Telit

Deux courriels envoyé par le support Telit suite a ma demande d'aide (eDRX, PSM, CFUN, DTR...)

De : TS-EMEA <ts-emea@telit.com>
Envoyé : jeudi 2 avril 2026 15:06
À : Lucas Del Cerro <lucas.delcerro@4mod.fr>
Cc : yann.lepine@telit.com; jerome.martin@telit.com
Objet : [4MOD] ME310M1-W1 - eDRX issue | Case Number: 00676947

Hello Lucas,

Thank you for your email.

On the module side, you did all the correct steps to enable the eDRX feature with the NW.

There is a remote case in which the NW confirms the possibility to use the eDRX feature (and so you see the timers with AT+CEDRXRDP), but at the same time doesn't allow it in the attach/registration preliminary phase. This is something pretty common in Europe, but for CAT1 and CAT4 technologies.

In CAT-M and NB-IoT, the eDRX feature is supported by almost all operators, so I guess the problem isn't here.

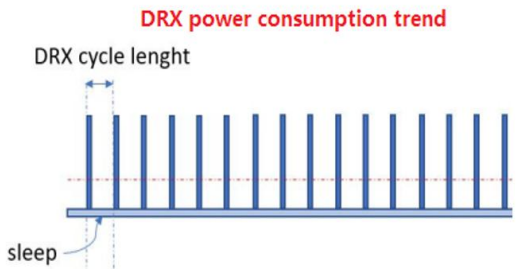
In case of further doubts, we could verify if the NW is really providing it by collecting a module trace from the AUX serial port.

What I suspect instead is that we need to interpret the results better.

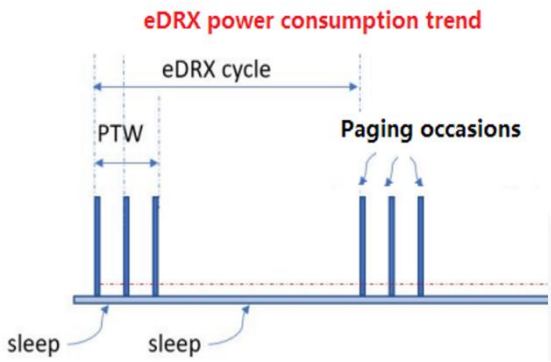
Let's recall some theoretical aspects:

When registered to the Network, the UE in IDLE mode checks the presence of new data for it at periodic time intervals defined by the DRX cycle assigned by the Network.

The instant at which the UE is in listening mode with the network is called “paging occasion”. Normally, the paging occasion cycle in IoT networks is equal to 1.28s or 2.56s.
The AVG power consumption in this scenario is around 8mA for the ME310M1:



eDRX is a service that can be provided by the Network - upon request of the UE – which allows the UE to alternate windows with normal DRX behavior (PTW), a sleep phases (UE registered, ready for Uplink traffic but isolated for Downlink traffic)



In terms of power consumption, the only difference is to have fewer of these peaks. The power consumption in the bottom (I mean the sleep phase of both scenarios) is the same. Since the peak consumption contribution is minimal compared to the bottom consumption, using eDRX alone never makes practical sense to reduce power consumption. It is always combined with some processor power-saving mode. These modes (set with CFUN) drastically reduce power consumption during the "sleep" phase. By reducing this, the reduction in paging occasions introduced by eDRX actually has a tangible and beneficial effect.

So, only activating the eDRX, it is expected that the power consumption is comparable to the "normal" consumption without it.

This is also the reason why you won't find the power consumption with only eDRX in the "Hardware Design Guide". It is always associated with a low power mode.

Now, the CFUNs you could use are three. On all of them, the power consumption will be only reduced in the IDLE phase (not during data traffic). A module in low power will have the AT interface disabled. It can be enabled when you want if you need to send AT commands. Let's see these <fun> states:

- CFUN: 0 -> Once sent, the module immediately enters low power. You can wake up the module with the RTS line. Otherwise, the UE will automatically wake up in case of active events from the NW (SMS, data, ..).

- CFUN: 5 --> It is equivalent to CFUN: 0 in terms of power consumption. The difference is that once sent, the module enters/exits from the LPM in function of the value of the DTR serial line. So the device won't automatically wake up from this CFUN state. Your application has full control

- CFUN: 9 --> It is a CFUN optimized for the use of the SIM Suspend feature. It is a feature that the SIM card could support and that allows the module to reach also few uA of AVG consumption. The UE wakes up from CFUN:9 with the WAKE0 signal or an active event from the NW.

Please remember not to mix CFUN with PSM. PSM is a service provided by the NW in which the module is turned OFF storing the NW information. This is a strategy to not repeat the attach procedure when the UE returns available.

In all the CFUN statuses, the module is always registered and is always available to receive/send data.

Attached, you can find a guide that can help you. Now I don't want to add too much information in a single email but you feel free to ask all the doubts you have.

Best regards,

Matteo Soccol

EMEA Technical Support
Telit Cinterion Application Engineering

De : TS-EMEA <ts-emea@telit.com>
Envoyé : jeudi 2 avril 2026 16:28
À : Lucas Del Cerro <lucas.delcerro@4mod.fr>
Cc : yann.lepine@telit.com; jerome.martin@telit.com
Objet : [4MOD] ME310M1-W1 - eDRX issue | Case Number: 00676947

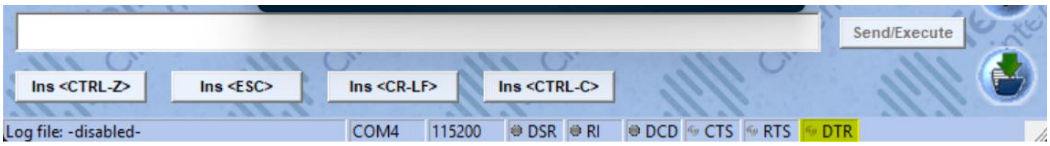
Hello Lucas,

Please note that from tomorrow to the 8th of April, the technical support in Italy will be off due to the company closure during the Easter holidays.
Below the answers for your questions:

1. Without a microcontroller available yet, is there a way to simulate the DTR control (for CFUN=5) directly from the Telit AT Controller? I would like to validate the CFUN=5

Sure, you can. You shall open the Telit AT controller with the "Manual - RTS/DTR" setting:

Then you can move the signal with a double click in the DTR icon in the bottom of the Telit AT Command window:



Remember to configure it in the proper way on the ME310M1: the DTR shall be mapped in an available GPIO (e.g. GPIO1 --> AT#GPIO=1,0,9) and then you need to physically connect this GPIO with the DTR of the serial port.

2. For CFUN=0, you mentioned the module can wake up automatically on network events (SMS, data). Does this mean that if the network sends a downlink packet during the eDRX sleep phase, the module will wake up by itself and process it ?without any hardware signal needed?

Exactly. The module will automatically wake up as soon as the URC linked to the event is generated. So, if you have, for example, a socket open, the incoming data will generate a RING URC. This URC will wake up the module that will automatically switch from CFUN:0 to CFUN:1. This means that you have to set the module again in low power with a new AT+CFUN=0 command (when desired). For SMSs, the URCs are enabled with the AT+CNMI command.

Please note that in CFUN:5 you could instead instruct the RING serial line to move in case of any incoming URC. So your microcontroller can wake up the module by moving the DTR line each time there is a variation of the RING, which works like an interrupt for you.

3. Regarding the WiFi scan feature available in the ME310M1 firmware: is there any known conflict or limitation when combining the WiFi scanning sequence (AT%WIFICMD="START") with CFUN=5 or CFUN=0 low power modes? For example, does the module need to be in CFUN=1 to perform the scan?

Yes, you need to be in CFUN: 1 (full operative mode) to run the WiFi scan. In any low-power mode forced by CFUN, you won't be able to send AT commands. In any case, CFUN is able to reduce the power consumption when the processor is in IDLE. During the WiFi scan, the processor isn't in IDLE, so it makes no sense to force it in low power. So you should alternate the low power phases with the phases where the WiFi scan is done.

4. Is there a recommended AT command sequence for a typical wake → scan → send → sleep cycle that you would suggest for a CAT-M IoT device using eDRX + CFUN?

You can simply keep the eDRX always enabled. Then, if you don't need to do any operation but only wait for a timer or incoming data, you can enter in low power. Each

time you need to use the module, you need to wake it up. How, It depends on the CFUN used. E.g. with CFUN:5

--> UE registered, DTR ON;

--> enable eDRX if it was disabled

--> AT+CFUN=5

Routine:

--> Do I need to do any action with the module (WiFi scanning, data traffic, SMSs, ..): keep the DTR ON to have control of the AT interface;

--> Do I need to wait for the next action to do? move the DTR to OFF to enter low power mode

--> Do I need to do an action in the AT interface? move the DTR to ON to take control of the AT interface and proceed with the action of interest.

Best regards,

Matteo Soccol

EMEA Technical Support
Telit Cinterion Application Engineering